

Introduction to Data Science and Engineering

- Data visualization



Some materials courtesy of
Rafael A. Irizarry, and are modified
from the original version.

Zhenqin (Michael) Wu / 吳楨欽

School of Computing and Data Science
University of Hong Kong

Slide deck originally created by RB Luo



In this lecture

- Data visualization and its importance
- `ggplot2` basics
- Visualize data distributions



In this lecture

- **Data visualization and its importance**
- ggplot2 basics
- Visualize data distributions

Data visualization in the early days

John Snow Cholera map

- Published in 1854
- Bar plots showing cholera death overlaid on a map

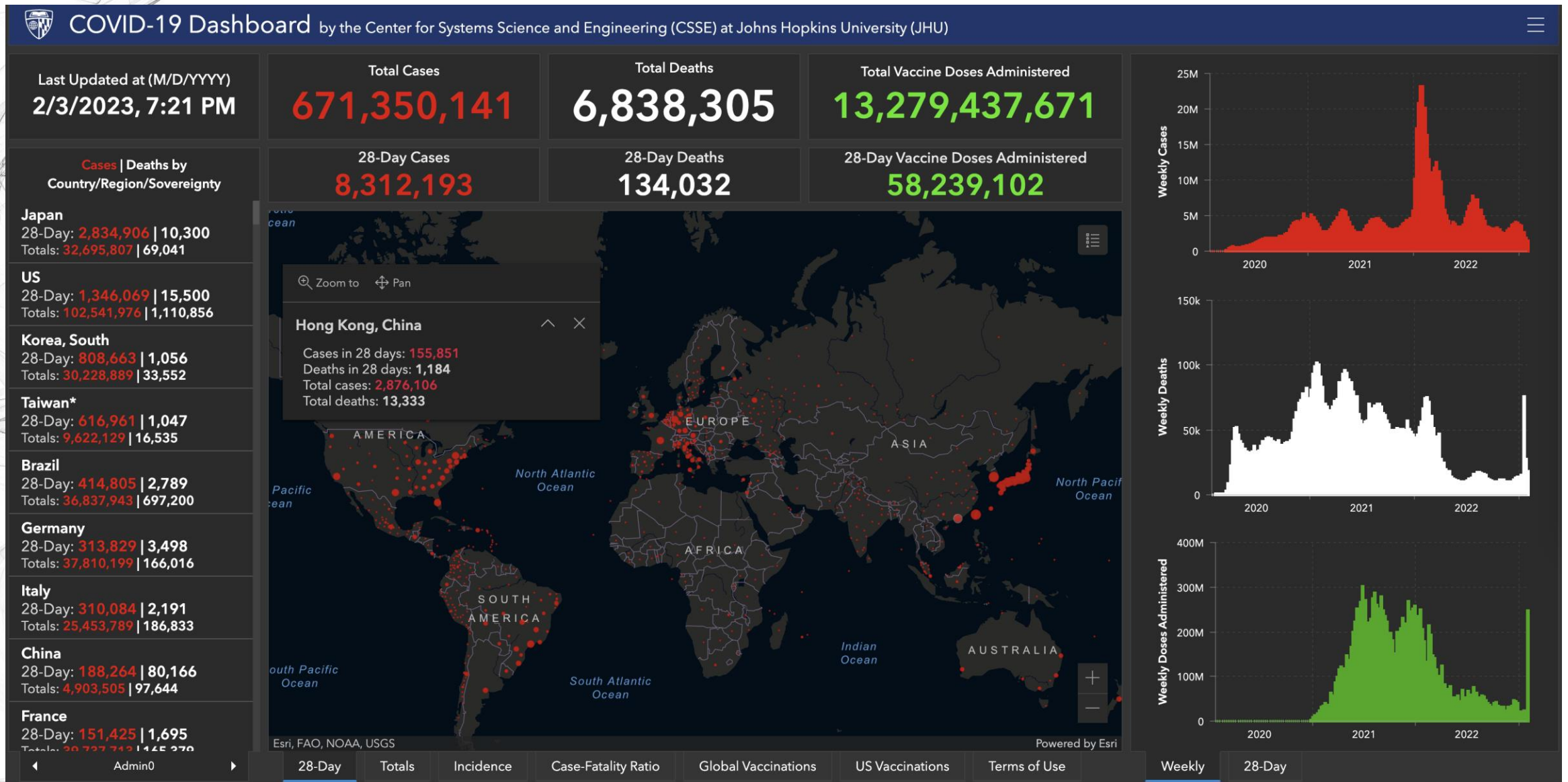
See also:

https://en.wikipedia.org/wiki/1854_Broad_Street_cholera_outbreak

<https://www.tableau.com/learn/articles/best-beautiful-data-visualization-examples>



Data visualization nowadays



See: <https://coronavirus.jhu.edu/map.html>

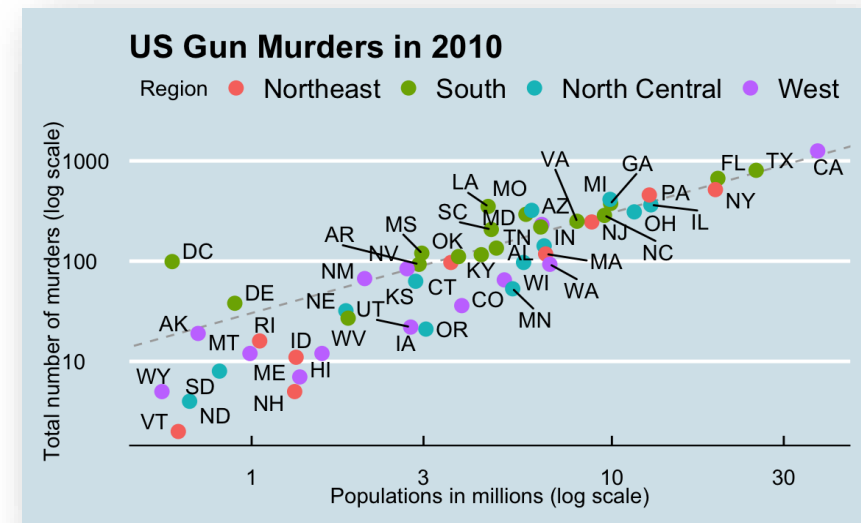
Why visualize?

Look at the table for 10 seconds, what do you see?

```
> library(dslabs)
> data(murders)
> head(murders)
```

	state	abb	region	population	total
1	Alabama	AL	South	4779736	135
2	Alaska	AK	West	710231	19
3	Arizona	AZ	West	6392017	232
4	Arkansas	AR	South	2915918	93
5	California	CA	West	37253956	1257
6	Colorado	CO	West	5029196	65

Look at a sensible visualization of the same table, what do you see?



Which states have the largest populations?

Which states have the smallest?

Is there a relationship between population size and total murders?

How do murder rates vary across regions of the country?

表 2.2 1991 年、1996 年、2001 年、2006 年、2011 年、2016 年及 2021 年按年齡組別劃分的人口百分比分布

Table 2.2 Percentage distribution of the population by age group, 1991, 1996, 2001, 2006, 2011, 2016 and 2021

年齡組別 Age group	人口百分比 Percentage distribution of population						
	1991	1996	2001	2006	2011	2016	2021
包括外籍家庭傭工 Including foreign domestic helpers							
0 – 14	20.8	18.7	16.4	13.7	11.6	11.3	10.9
15 – 24	15.2	14.3	13.6	13.3	12.4	10.7	8.0
25 – 54	46.4	48.6	51.3	50.9	49.6	46.9	44.8
55 – 64	8.8	8.2	7.5	9.7	13.0	15.2	16.8
65+	8.7	10.2	11.2	12.4	13.3	15.9	19.6
年齡中位數（歲） Median age (years)	31.6	34.2	36.8	39.6	41.7	43.4	46.3
不包括外籍家庭傭工 Excluding foreign domestic helpers							
0 – 14	21.1	19.2	16.9	14.2	12.1	11.8	11.4
15 – 24	15.2	14.3	13.5	13.3	12.7	11.1	8.2
25 – 54	45.9	47.8	50.2	49.7	47.8	44.7	42.6
55 – 64	8.9	8.4	7.7	10.0	13.5	15.8	17.4
65+	8.9	10.4	11.6	12.9	13.9	16.6	20.5
年齡中位數（歲） Median age (years)	31.6	34.4	37.2	40.1	42.4	44.3	47.3

表 2.4 1991 年、1996 年、2001 年、2006 年、2011 年、2016 年及 2021 年按年齡組別劃分的性別比率

Table 2.4 Sex ratio by age group, 1991, 1996, 2001, 2006, 2011, 2016 and 2021

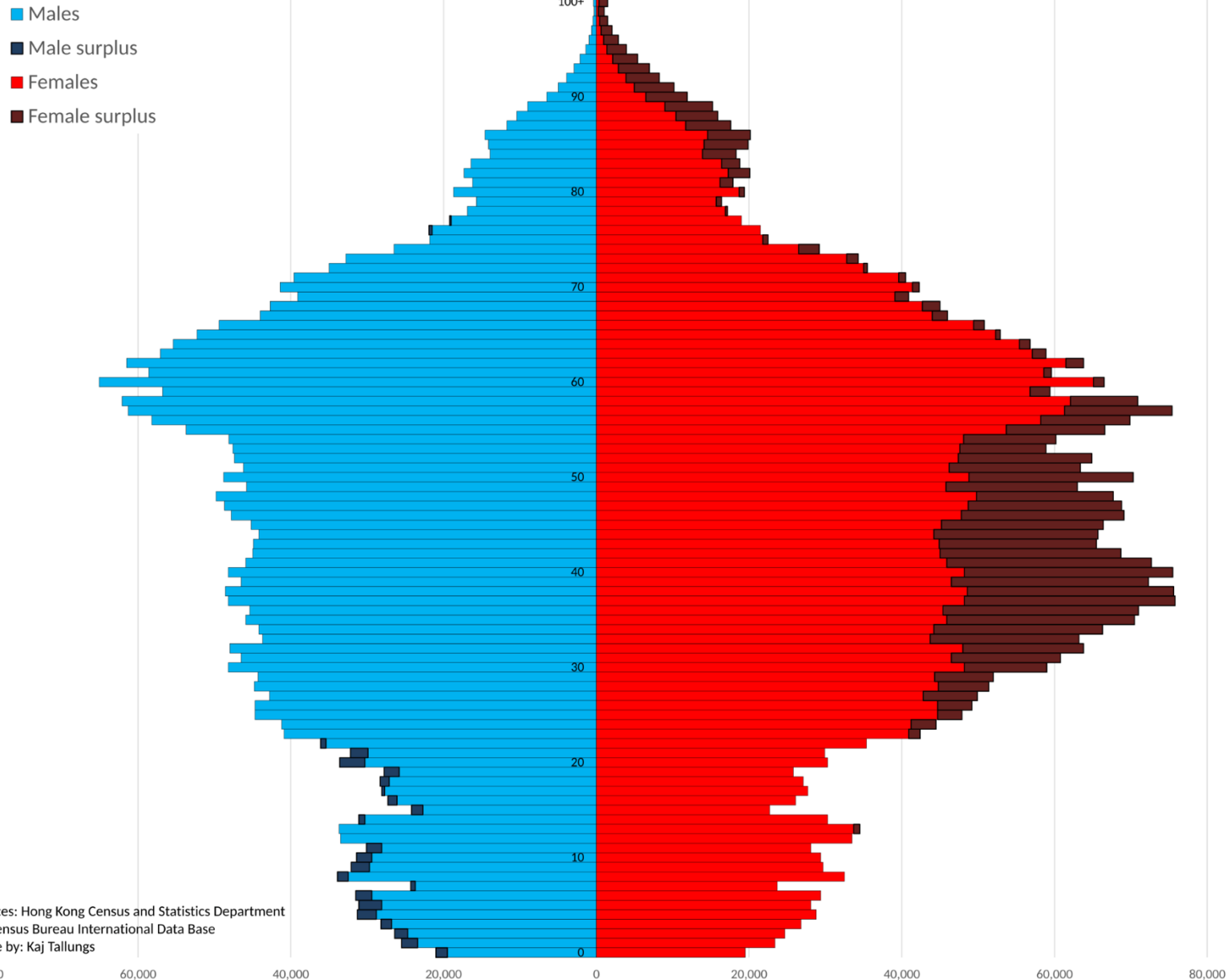
年齡組別 Age group	性別比率 ⁽¹⁾ Sex ratio ⁽¹⁾						
	1991	1996	2001	2006	2011	2016	2021
包括外籍家庭傭工 Including foreign domestic helpers							
0 – 14	1 082	1 075	1 072	1 063	1 073	1 065	1 062
15 – 24	1 042	1 023	989	972	1 006	1 019	1 028
25 – 34	999	886	814	786	724	735	815
35 – 44	1 096	1 006	908	793	725	682	666
45 – 54	1 225	1 146	1 037	974	892	787	726
55 – 64	1 101	1 139	1 154	1 056	990	971	887
65 – 74	907	941	1 004	1 022	1 052	1 003	955
75+	599	647	668	687	723	750	796
合計 Overall	1 044	1 001	956	912	876	852	839
不包括外籍家庭傭工 Excluding foreign domestic helpers							
0 – 14	1 082	1 075	1 072	1 063	1 073	1 065	1 062
15 – 24	1 072	1 079	1 074	1 039	1 043	1 044	1 049
25 – 34	1 072	1 002	974	958	940	921	973
35 – 44	1 146	1 075	996	869	836	839	850
45 – 54	1 247	1 168	1 076	1 011	933	841	792
55 – 64	1 102	1 143	1 162	1 065	1 002	983	904
65 – 74	907	941	1 004	1 022	1 053	1 004	956
75+	599	647	668	687	723	750	796
合計 Overall	1 074	1 048	1 021	971	948	925	910

From: Demographic Trends in Hong Kong
<https://www.censtatd.gov.hk/en/EIndexbySubject.html?pcode=B1120017&scode=160>

註釋： (1) 指男性數目相對每千名女性的比率。

Note : (1) Refers to the number of males per 1 000 females.

Hong Kong, December 2020



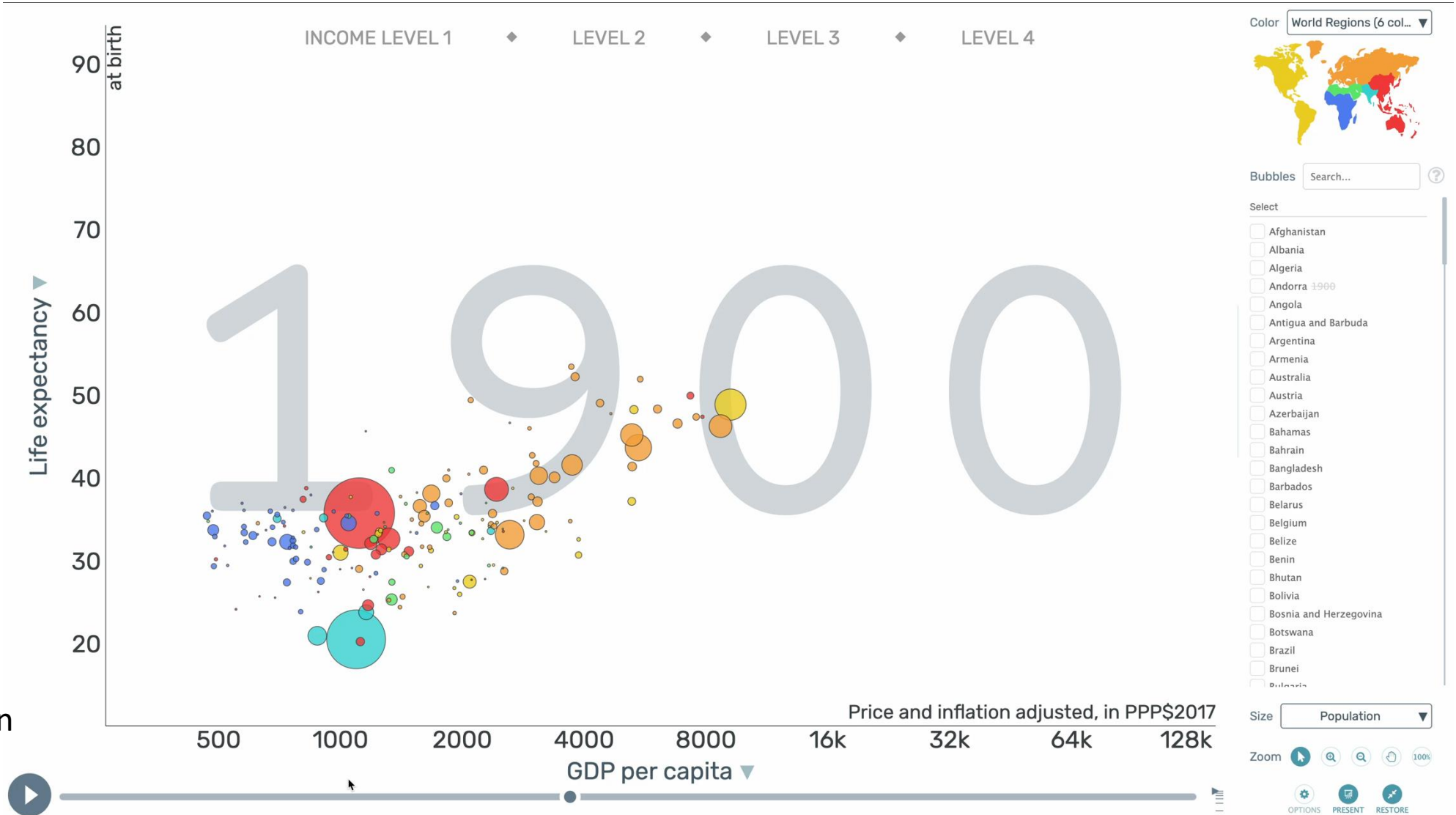
Why data visualization is helpful?

- Simplifies complex data: more intuitive understanding of data trends;
- Direct comparisons: more discoveries which would otherwise be missed;
- Enhances communication: graph speaks louder than numbers.

Data visualization is the strongest tool of what we call **exploratory data analysis** (EDA).

“The greatest value of a picture is when it forces us to notice what we never expected to see.” --- John W. Tukey

The different axes of data visualization



See also: https://www.ted.com/talks/hans_rosling_the_best_stats_you_ve_ever_seen
[https://www.gapminder.org/tools/#\\$chart-type=bubbles&url=v2](https://www.gapminder.org/tools/#$chart-type=bubbles&url=v2)

“Garbage in, garbage out”

- Mistakes, biases, systematic errors and other unexpected problems often lead to falsified data, which should be handled with extra care. Failure to discover these problems can give rise to flawed analyses and false discoveries.

Mars Climate Orbiter Crash: The spacecraft went into a trajectory that comes too close to Mars, and it was destroyed in the atmosphere. An investigation attributed the failure to a measurement mismatch between two measurement systems: *SI units (metric)* by NASA and *US customary units* by Lockheed Martin.

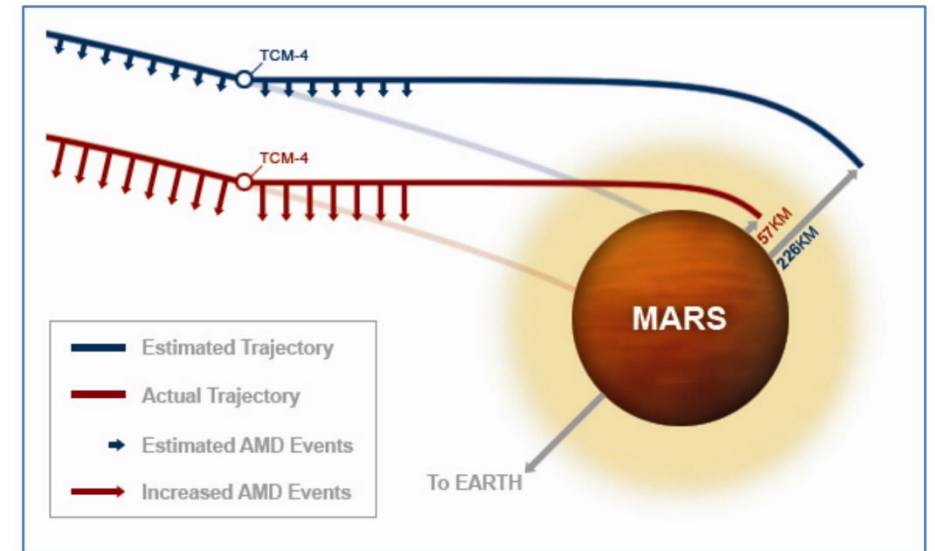
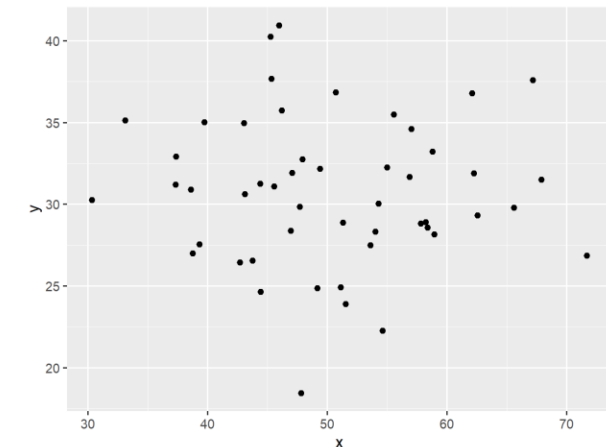
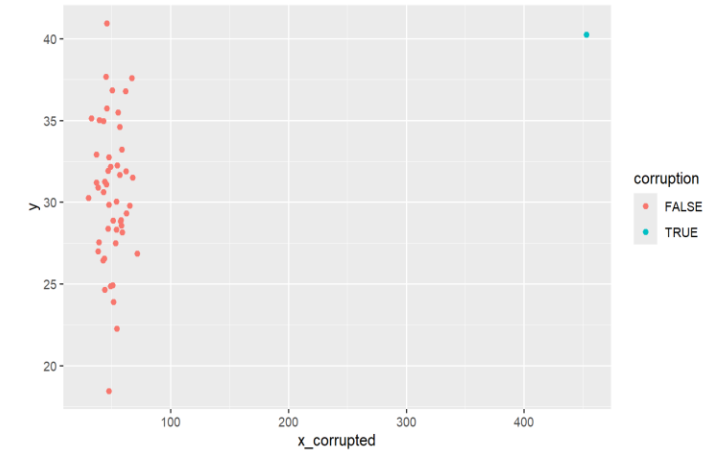


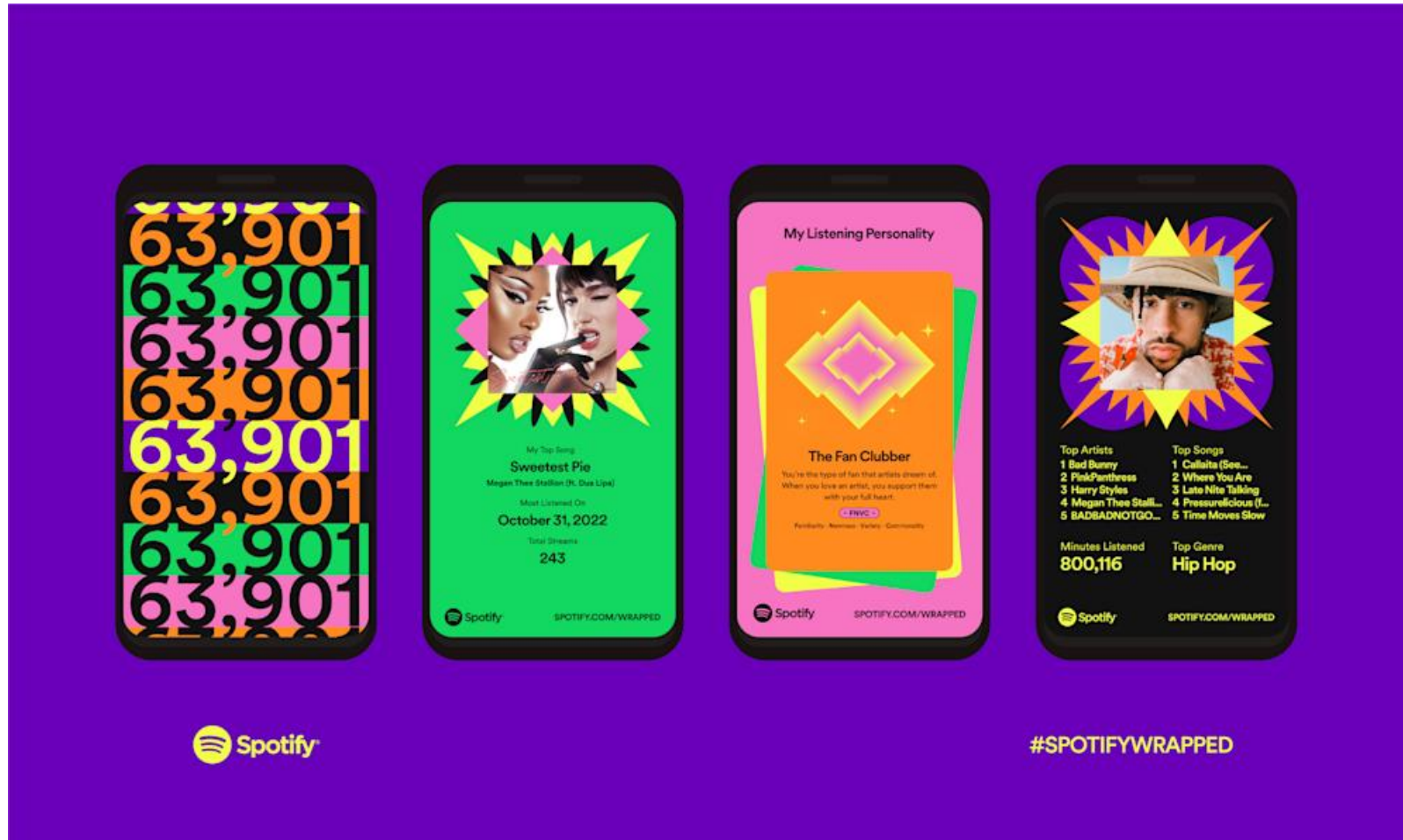
Figure 2: Actual vs. estimated approach to Mars (not to scale).

“Garbage in, garbage out”

```
> library(tidyverse)
> set.seed(123)
>
> x <- rnorm(50, mean = 50, sd = 10) # Random normal data
> y <- rnorm(50, mean = 30, sd = 5)  # Random normal data
> data <- data.frame(x, y)
>
> original_correlation <- cor(x, y)
> cat("Original correlation:", original_correlation, "\n")
Original correlation: -0.03586983
>
> x[20] <- x[20] * 10 # Mistakenly put the digit
> data["corruption"] <- rep(FALSE, length(x))
> data[20, "corruption"] <- TRUE
> data["x_corrupted"] <- x
> corrupted_correlation <- cor(x, y)
> cat("Corrupted correlation:", corrupted_correlation, "\n")
Corrupted correlation: 0.2975772
```



Data visualization is everywhere

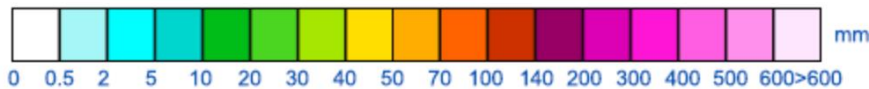
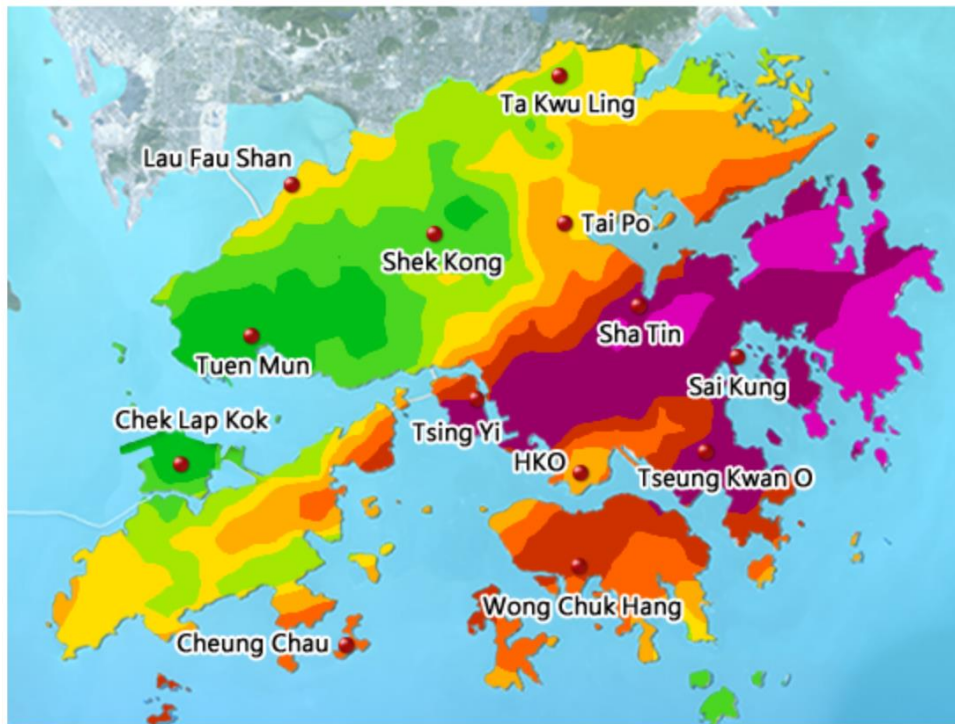


See also: <https://www.dandad.org/awards/professional/2025/240901/10-years-of-wrapped/>

Data visualization is everywhere

Yellow/Red rainstorm

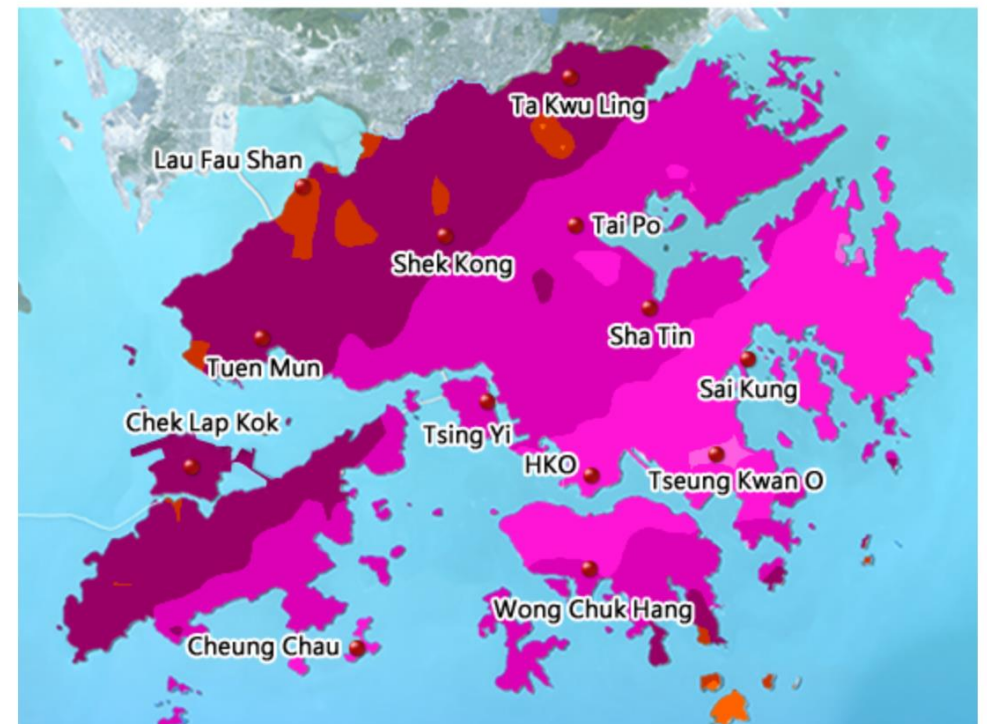
Total rainfall on 4-Aug-2025 (based on raingauges and radar data)



香港天文台
HONG KONG OBSERVATORY

Black rainstorm

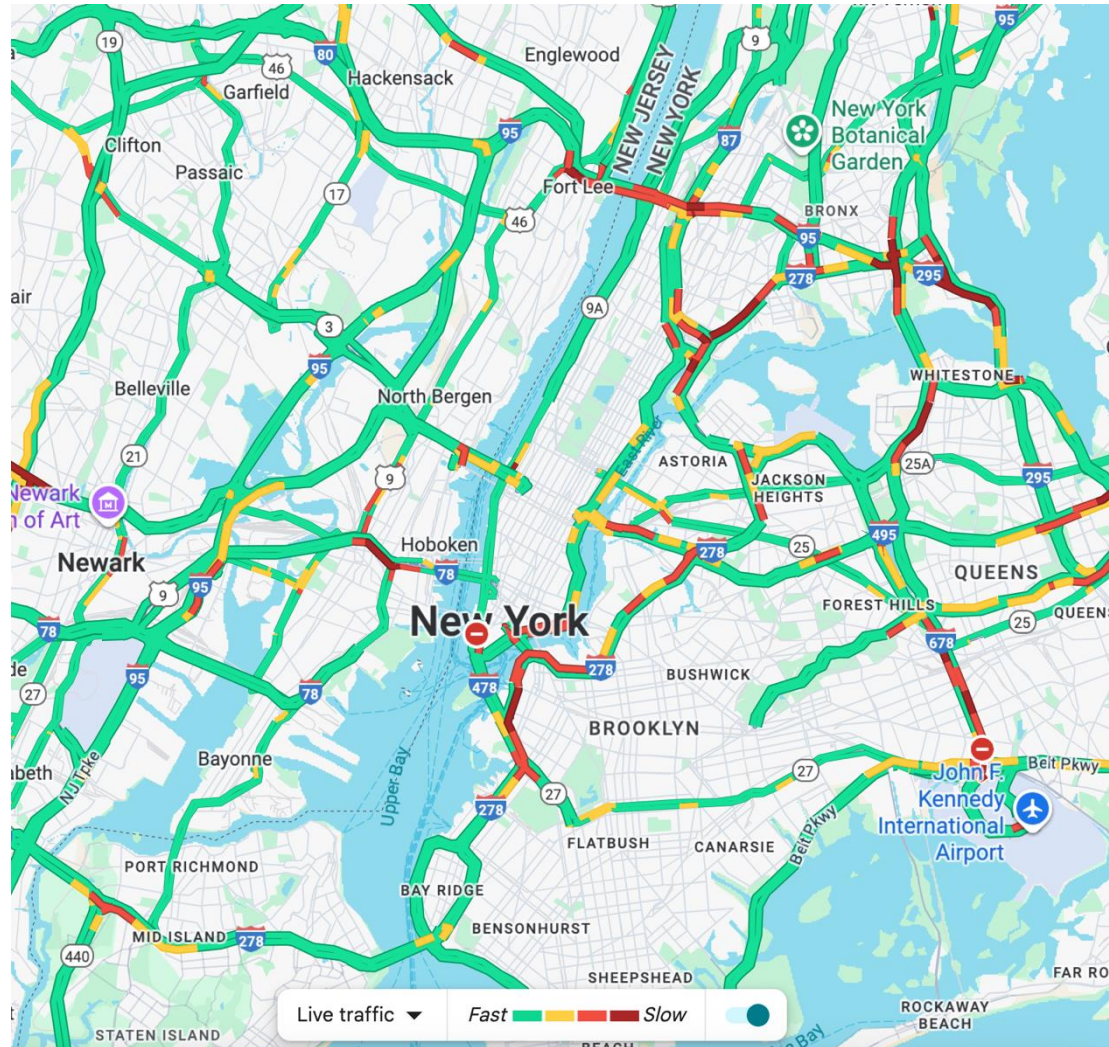
Total rainfall on 5-Aug-2025 (based on raingauges and radar data)



香港天文台
HONG KONG OBSERVATORY

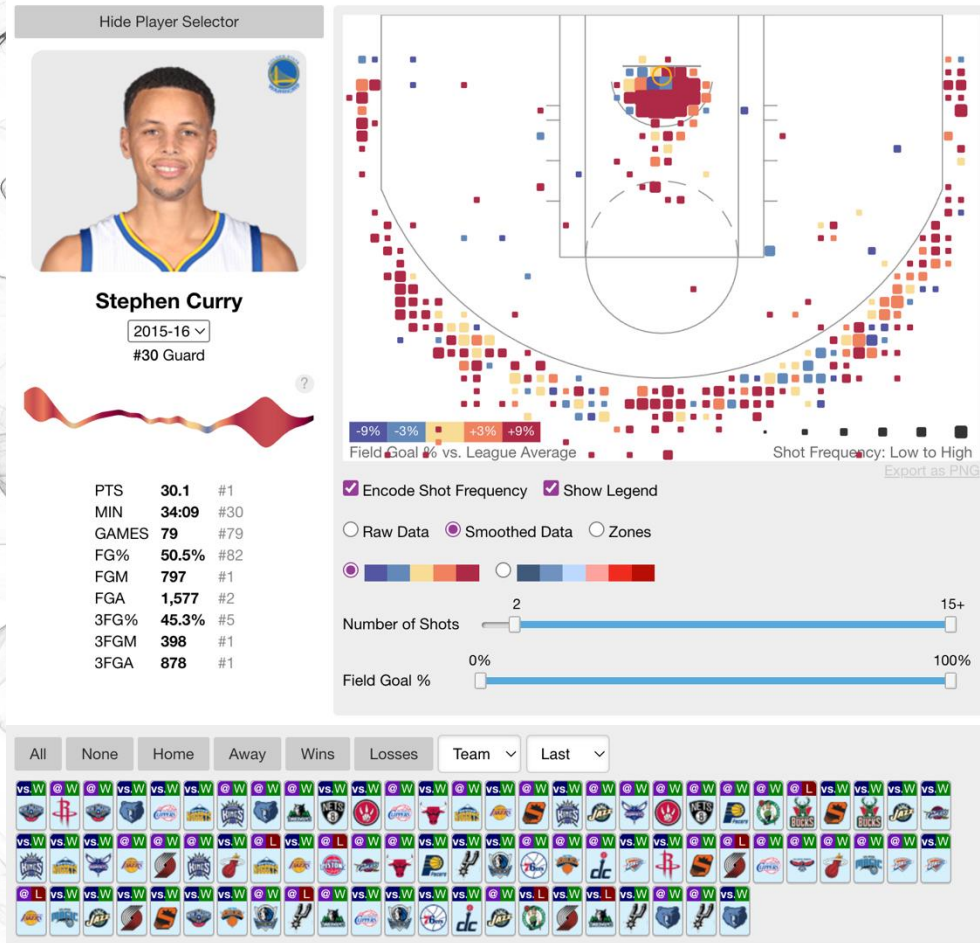
https://www.hko.gov.hk/en/wxinfo/rainfall/isohyet_daily.shtml

Data visualization is everywhere

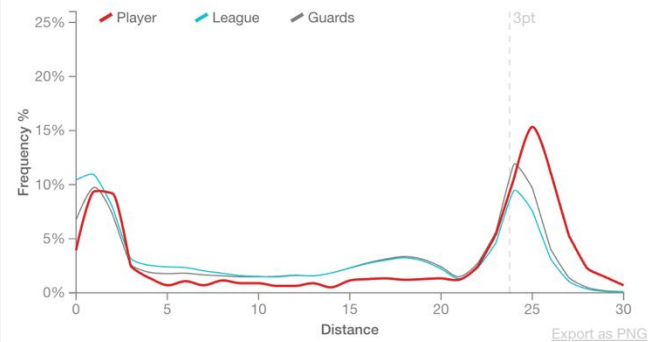


New York @ 9:30AM

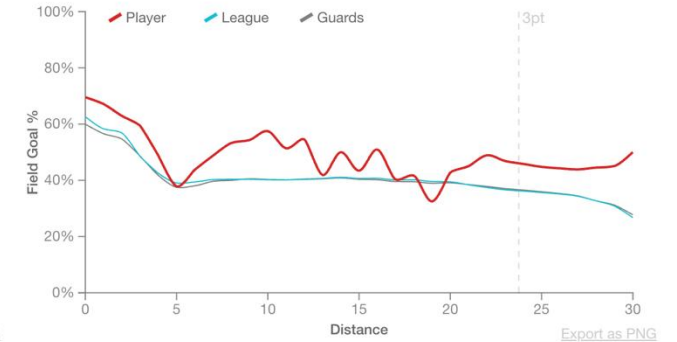
Data visualization is everywhere



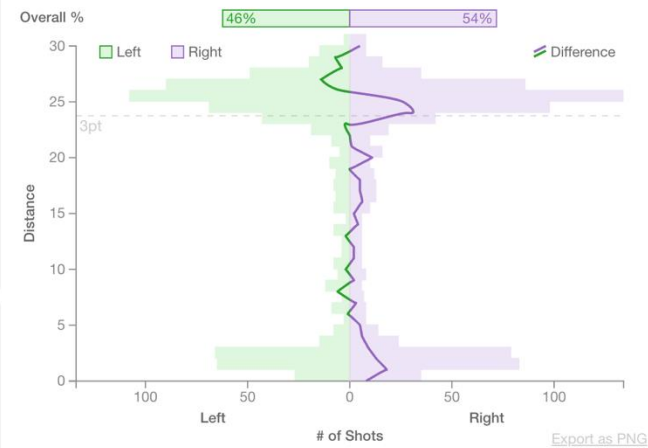
Shot Frequency % by Distance



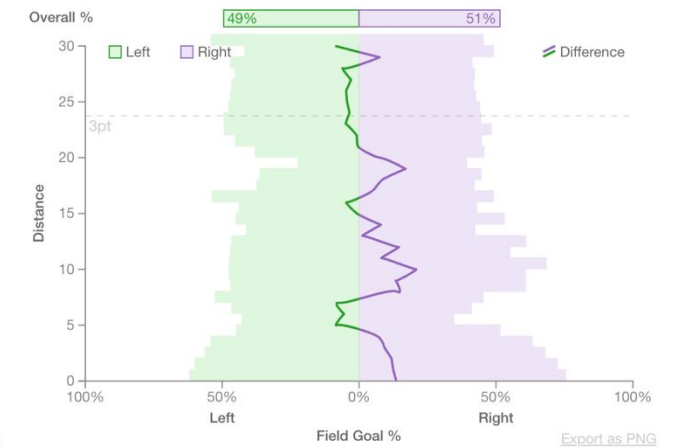
Field Goal % by Distance



Shot Frequency: Left Side vs. Right Side



Field Goal %: Left Side vs. Right Side





In this lecture

- Data visualization and its importance
- **ggplot2 basics**
- Visualize data distributions



ggplot2

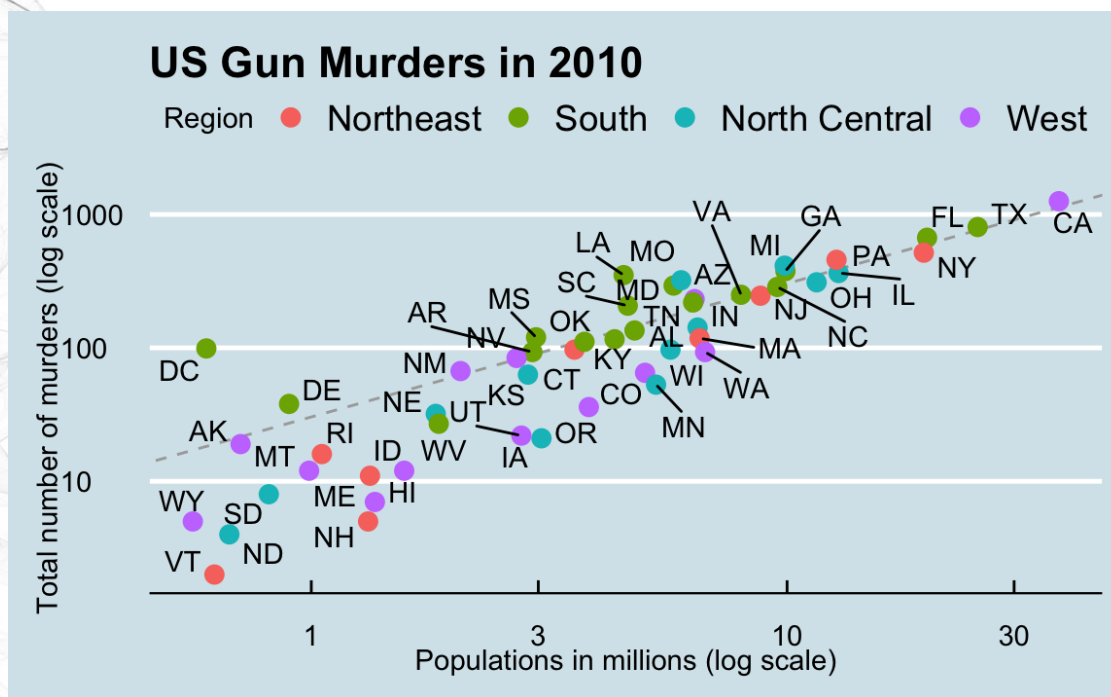
```
> library(ggplot2)
```

or simply

```
> library(tidyverse)
```

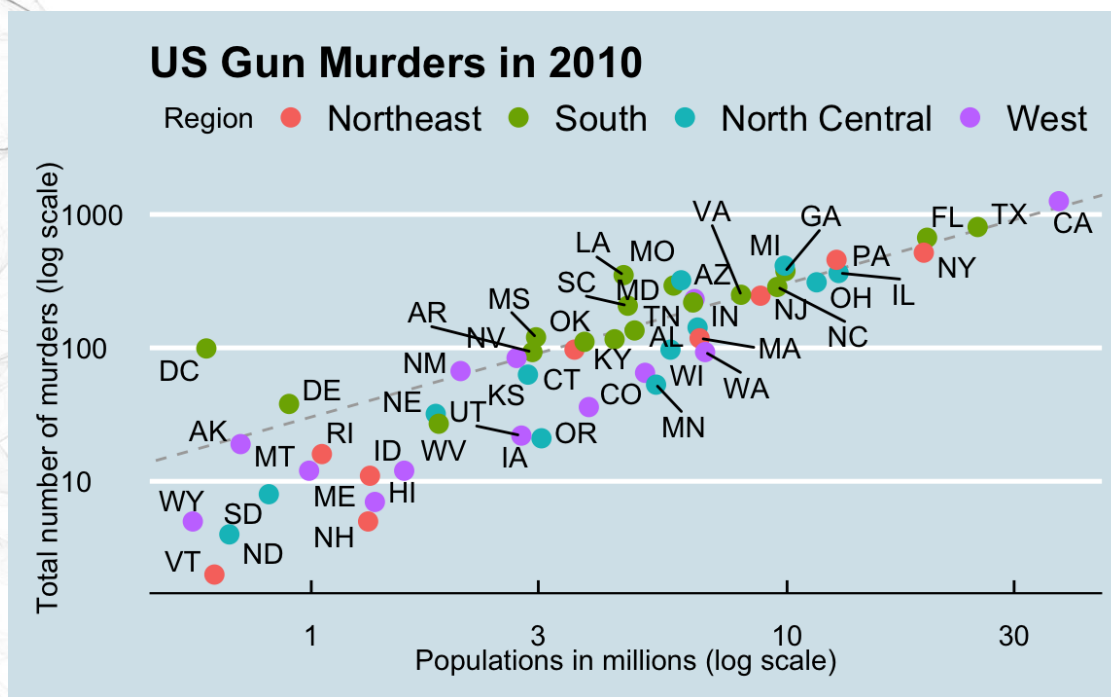
- Facts about `ggplot2`
 - `gg` means "grammar of graphics";
 - Like a lego system: by learning a handful of `ggplot2` building blocks and its grammar, you will be able to create hundreds of different plots.
 - `ggplot2` generates layered plot: each layer contains different components, allowing custom arrangements of components or elements to a figure, one at a time.
- Other libraries in R that create graphics
 - `grid` and `lattice`
- `ggplot2` functions and arguments needs memorization, we recommend you have the `ggplot2` cheat sheet handy
 - <https://posit.co/wp-content/uploads/2022/10/data-visualization-1.pdf>
 - Search for "ggplot2 cheat sheet".

Components of a graph



- `ggplot2` has a layered design. What are the layers/components in the example graph?
 - **Data**: The US murders data table. This is termed the “data” component.
 - **Geometry**/type of plot: Scatter plot. This is termed the “geometry” component. Other geometries: bar plot, histogram, smooth densities, box plot, qqplot, violin plot, etc.
 - **Aesthetic**/axes: mapping between visual cues and data information. This plot uses multiple visual cues including x-axis, y-axis, and colors. What information does each cue represent?
 - **Other layers**:
 - Coordinate system
 - Title
 - x-ticks, y-ticks, x-label, y-label
 - Legend
 - Caption
 - ...

Components of a graph

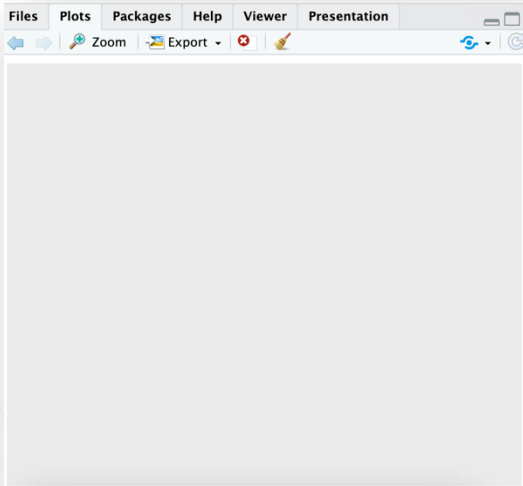


ggplot objects

```
> ggplot(data = murders)
```

or

```
> murders |> ggplot()
```



```
> p <- ggplot(data = murders)
> class(p)
[1] "gg"      "ggplot"
> p
> print(p)
```

- The first step is to create a `ggplot` object. We do this with the function `ggplot`, which initializes the graph.
 - Serves as the foundation of all following layers
 - The first argument is used to specify what data should be used, very handy for piping.
- It renders a plot, in this case a blank slate since no geometry has been defined. The only component we see is a grey background
- Try evaluate it or print it?



Geometries

- General syntax: creating a graph by adding layers. To do that:

```
DATA |> ggplot() + LAYER 1 + LAYER 2 + ... + LAYER N
```

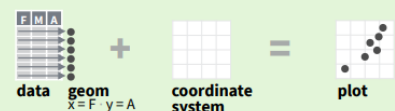
- Layers are used to define geometries, compute summary statistics, define what scales to use, or even change styles.

Data visualization with ggplot2: Available geometries: CHEATSHEET

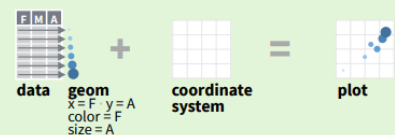


Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data set**, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

```
ggplot(data = <DATA>) +  
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>),  
  stat = <STAT>, position = <POSITION>) +  
  <COORDINATE_FUNCTION> +  
  <FACET_FUNCTION> +  
  <SCALE_FUNCTION> +  
  <THEME_FUNCTION>
```

required

Not required, sensible defaults supplied

ggplot(data = mpg, aes(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5" x 5" file named "plot.png" in working directory. Matches file type to file extension.

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

size - integer (in mm for size of points and text)

linewidth - integer (in mm for widths of lines)

shape - integer/shape name or a single character ("a")

0 1 2 3 4 5 6 7 8 9 10 11 12
□ ○ △ × ◇ ☆ * ✱ ✸ ✹
13 14 15 16 17 18 19 20 21 22 23 24 25
⊠ ⊡ ⊢ ⊣ ⊤ ⊥ ⊦ ⊧ ⊨ ⊩

Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

a <- ggplot(economics, aes(date, unemployment))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank() and **a + expand_limits()**
Ensure limits include values across all plots.

b + geom_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, yend, y, xend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend = "butt", linejoin = "round", linemitre = 1) - x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin = unemployment - 900, ymax = unemployment + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept = 0, slope = 1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); **c2** <- ggplot(mpg)

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly()
x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5)
x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy))
x, y, alpha, color, fill, linetype, size, weight

discrete

d <- ggplot(mpg, aes(fl))

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES

both continuous
e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point()
x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile()
x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl")
x, y, alpha, color, linetype, size

e + geom_smooth(method = lm)
x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

one discrete, one continuous

f <- ggplot(mpg, aes(class, hwy))

f + geom_col()
x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot()
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center")
x, y, alpha, color, fill, group

f + geom_violin(scale = "area")
x, y, alpha, color, fill, group, linetype, size, weight

both discrete

g <- ggplot(diamonds, aes(cut, color))

g + geom_count()
x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

THREE VARIABLES

sealsSz <- with(seals, sqrt(delta_long^2 + delta_lat^2)); **l** <- ggplot(seals, aes(long, lat))

l + geom_contour(aes(z = z))
x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled(aes(fill = z))
x, y, alpha, color, fill, group, linetype, size, subgroup

continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d()
x, y, alpha, color, group, linetype, size

h + geom_hex()
x, y, alpha, color, fill, size

continuous function

i <- ggplot(economics, aes(date, unemployment))

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))

j + geom_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() - x, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh**()

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

Draw the appropriate geometric object depending on the simple features present in the data. **aes()** arguments: **map_id**, **alpha**, **color**, **fill**, **linetype**, **linewidth**.

nc <- sf::st_read(system.file("shape/nc.shp", package = "sf"))

ggplot(nc) + geom_sf(aes(fill = AREA))

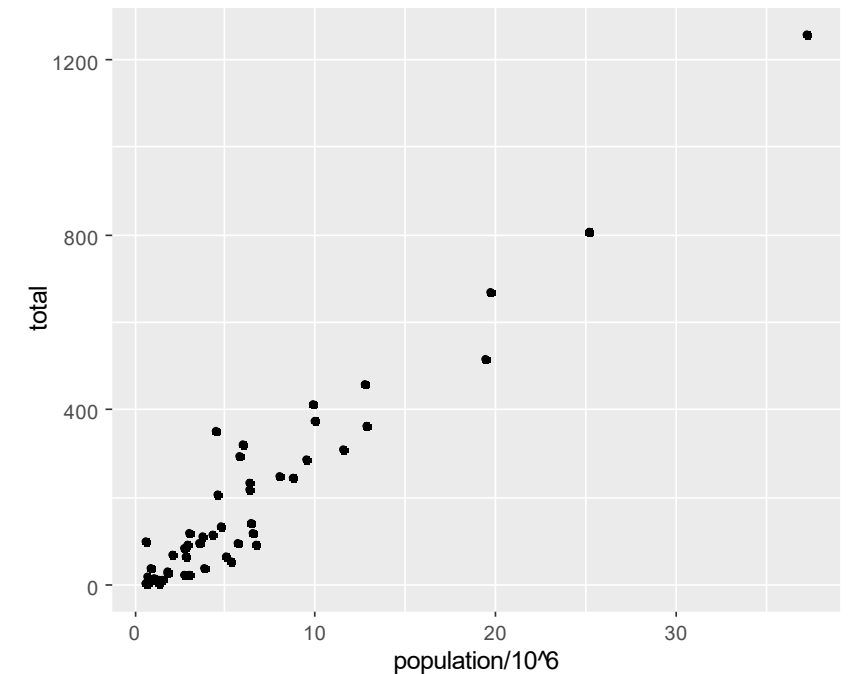
Aesthetic mappings

Aesthetic mappings describe how data contents are mapped to graph components, such as position on an axis, size, or color. Use the `aes` function as an argument to the geometries to achieve that. This example produces a scatterplot of total murders versus population in millions

```
> murders |> ggplot() +  
+ geom_point(aes(x = population/10^6, y = total))
```

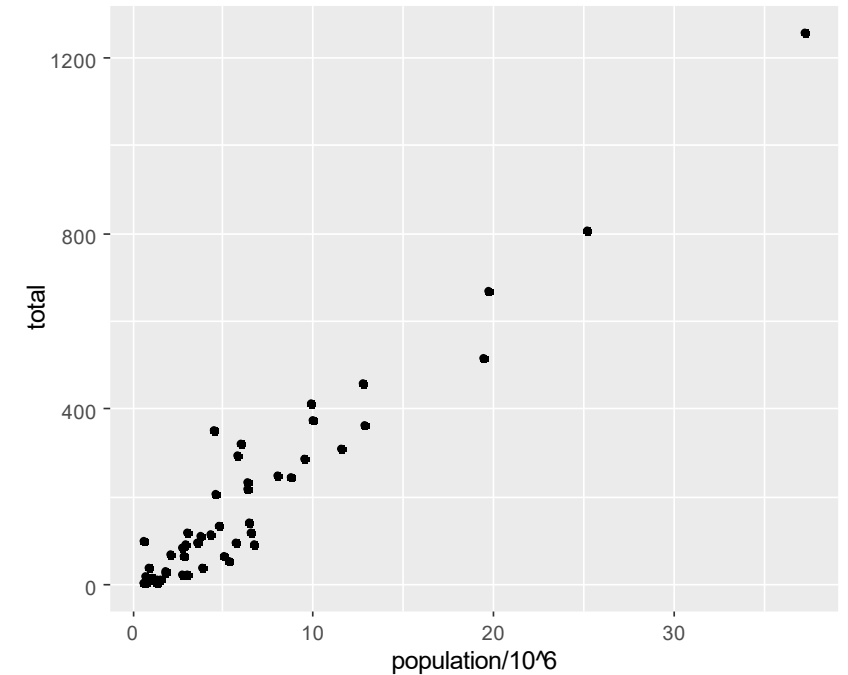
- `x` and `y` are always the first two arguments.
- We can simply add a layer to the `p` object that was defined earlier.

```
> p <- ggplot(data = murders)  
> p <- p + geom_point(aes(population/10^6, total))  
> p
```



Aesthetic mappings

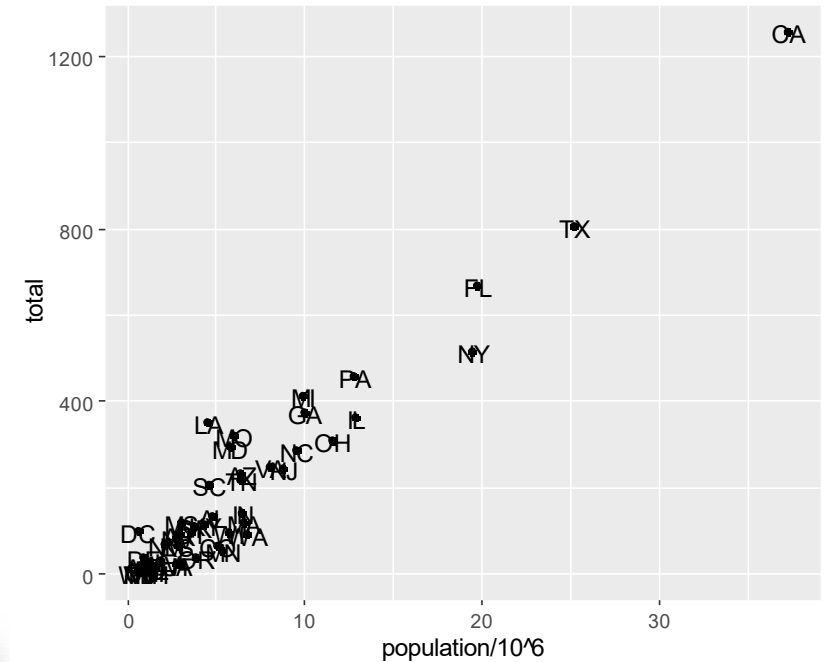
- **The scale and labels are defined by default**
- As part of `tidyverse`, the mapping `aes` also uses the variable/column names from the data component
- The behavior of recognizing the variables from the data component is quite specific to `aes`. With most functions, if you try to access the values of `population` or `total` outside of `aes` you receive an error



Add another layer

- We add a second layer to the plot: labeling each point to identify the state it represents.
- The `geom_label` and `geom_text` functions allow us to add texts to the plot: with or without a rectangle behind the text, respectively.
- The `label` argument in the aesthetic mapping specifies which text to use for labeling each point.

```
> ggplot(data = murders) +  
+ geom_point(aes(population/10^6, total)) +  
+ geom_text(aes(population/10^6, total, label = abb))
```



Data visualization with ggplot2 : : CHEATSHEET



Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data set**, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

ggplot (data = <DATA>) +
<GEOM_FUNCTION> (mapping = aes(<MAPPINGS>),
stat = <STAT>, position = <POSITION>) +
<COORDINATE_FUNCTION> +
<FACET_FUNCTION> +
<SCALE_FUNCTION> +
<THEME_FUNCTION>

required
Not required, sensible defaults supplied

ggplot(data = mpg, aes(x = cty, y = hwy)) Begins a plot that you finish by adding layers to. Add one geom function per layer.

last_plot() Returns the last plot.

ggsave("plot.png", width = 5, height = 5) Saves last plot as 5" x 5" file named "plot.png" in working directory. Matches file type to file extension.

Aes

Common aesthetic values.

color and **fill** - string ("red", "#RRGGBB")

linetype - integer or string (0 = "blank", 1 = "solid", 2 = "dashed", 3 = "dotted", 4 = "dotdash", 5 = "longdash", 6 = "twodash")

size - integer (in mm for size of points and text)

linewidth - integer (in mm for widths of lines)

shape - integer/shape name or a single character ("a")

0 1 2 3 4 5 6 7 8 9 10 11 12
□ ○ △ × ◇ ☆ * ✱ ✸ ✹
13 14 15 16 17 18 19 20 21 22 23 24 25
⊠ ⊡ ⊢ ⊣ ⊤ ⊥ ⊦ ⊧ ⊨ ⊩

Geoms

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

a <- ggplot(economics, aes(date, unemployment))
b <- ggplot(seals, aes(x = long, y = lat))

a + geom_blank() and **a + expand_limits()**
Ensure limits include values across all plots.

b + geom_curve(aes(yend = lat + 1, xend = long + 1), curvature = 1) - x, yend, y, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend = "butt", linejoin = "round", linemitre = 1) - x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(alpha = 50)) - x, y, alpha, color, fill, group, subgroup, linetype, size

b + geom_rect(aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin = unemployment - 900, ymax = unemployment + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS

common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept = 0, slope = 1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend = lat + 1, xend = long + 1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous

c <- ggplot(mpg, aes(hwy)); **c2** <- ggplot(mpg)

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly()
x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5)
x, y, alpha, color, fill, linetype, size, weight

c2 + geom_qq(aes(sample = hwy))
x, y, alpha, color, fill, linetype, size, weight

discrete

d <- ggplot(mpg, aes(fl))

d + geom_bar()
x, alpha, color, fill, linetype, size, weight

TWO VARIABLES both continuous

e <- ggplot(mpg, aes(cty, hwy))

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_point()
x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile()
x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl")
x, y, alpha, color, linetype, size

e + geom_smooth(method = "lm")
x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1) - x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

continuous bivariate distribution

h <- ggplot(diamonds, aes(carat, price))

h + geom_bin2d(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

h + geom_density_2d()
x, y, alpha, color, group, linetype, size

h + geom_hex()
x, y, alpha, color, fill, size

continuous function

i <- ggplot(economics, aes(date, unemployment))

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error

df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))

j + geom_crossbar(fatten = 2) - x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar() - x, ymax, ymin, alpha, color, group, linetype, size, width
Also **geom_errorbarh**()

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange() - x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps

Draw the appropriate geometric object depending on the simple features present in the data. **aes()** arguments: **map_id**, **alpha**, **color**, **fill**, **linetype**, **linewidth**.

nc <- sf::st_read(system.file("shape/nc.shp", package = "sf"))

ggplot(nc) + geom_sf(aes(fill = AREA))

one discrete, one continuous

f <- ggplot(mpg, aes(class, hwy))

f + geom_col()
x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot()
x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center")
x, y, alpha, color, fill, group

f + geom_violin(scale = "area")
x, y, alpha, color, fill, group, linetype, size, weight

both discrete

g <- ggplot(diamonds, aes(cut, color))

g + geom_count()
x, y, alpha, color, fill, shape, size, stroke

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

THREE VARIABLES

sealsSz <- with(seals, sqrt(delta_long^2 + delta_lat^2)); **l** <- ggplot(seals, aes(long, lat))

l + geom_contour(aes(z = z))
x, y, z, alpha, color, group, linetype, size, weight

l + geom_contour_filled(aes(fill = z))
x, y, alpha, color, fill, group, linetype, size, subgroup

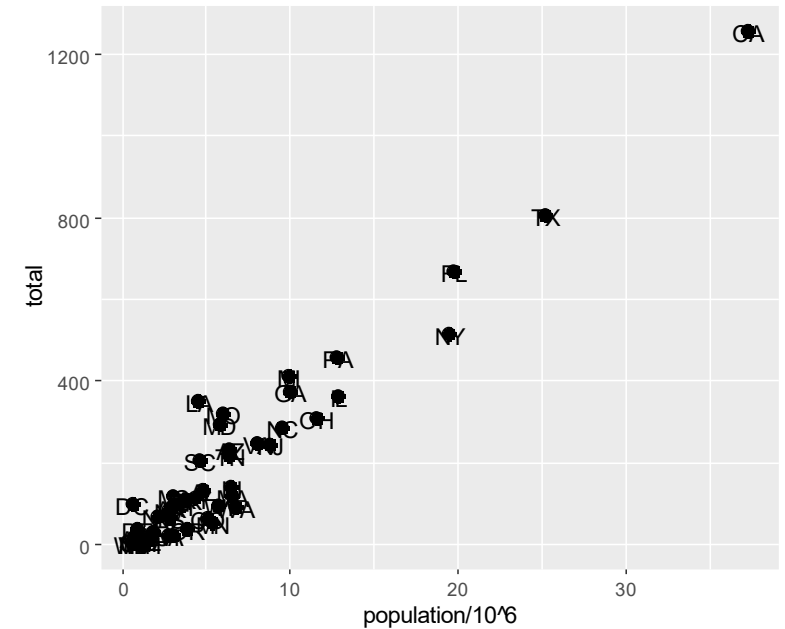
l + geom_raster(aes(fill = z), hjust = 0.5, vjust = 0.5, interpolate = FALSE)
x, y, alpha, fill

l + geom_tile(aes(fill = z))
x, y, alpha, color, fill, linetype, size, width

Tinkering with arguments

- Each geometry function has additional arguments to specify the details or styles of the plot.
- To make the point size larger:

```
> ggplot(data = murders) +  
+ geom_point(aes(population/10^6, total), size = 3) +  
+ geom_text(aes(population/10^6, total, label = abb))
```

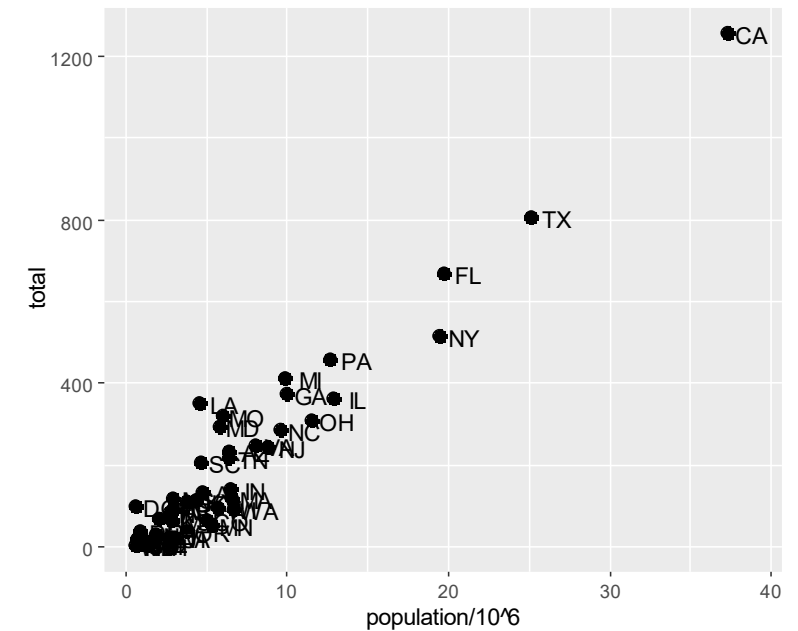


**Size is not data point-dependent,
and applies to all data points**

Nudge the label

- Now because the points are larger it is hard to see the labels. We can use the `nudge_x` argument, which moves the text slightly to the right or to the left

```
> ggplot(data = murders) +  
  + geom_point(aes(population/10^6, total), size = 3) +  
  + geom_text(aes(population/10^6, total, label = abb), nudge_x = 1.5)
```



Again, this is not data point-dependent.

Use global aesthetic mappings

```
> ggplot(data = murders) +  
+ geom_point(aes(population/10^6, total), size = 3) +  
+ geom_text(aes(population/10^6, total, label = abb), nudge_x = 1.5)
```

In the previous example, we defined the mapping `aes (population/10^6, total)` separately for the two geometries. We can avoid this by using a global aesthetic mapping

```
> ggplot(murders, aes(population/10^6, total, label = abb)) +  
+ geom_point(size = 3) +  
+ geom_text(nudge_x = 1.5)
```

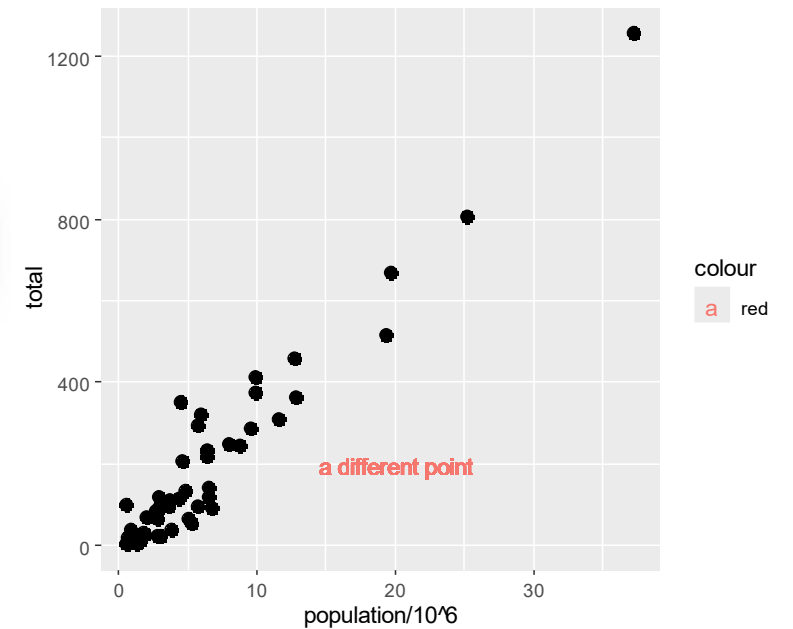


The global aesthetic mapping will be automatically applied to the following geometries. Non-relevant items (`label` for `geom_point`) will be ignored.

Use both global and local aesthetic mappings

- If necessary, we can override the global mapping with local mapping defined within each geometry.

```
> murders |> ggplot(aes(population/10^6, total)) +  
+   geom_point(size=3) +  
+   geom_text(aes(x=20, y=200, label="a different point", color="red"))
```

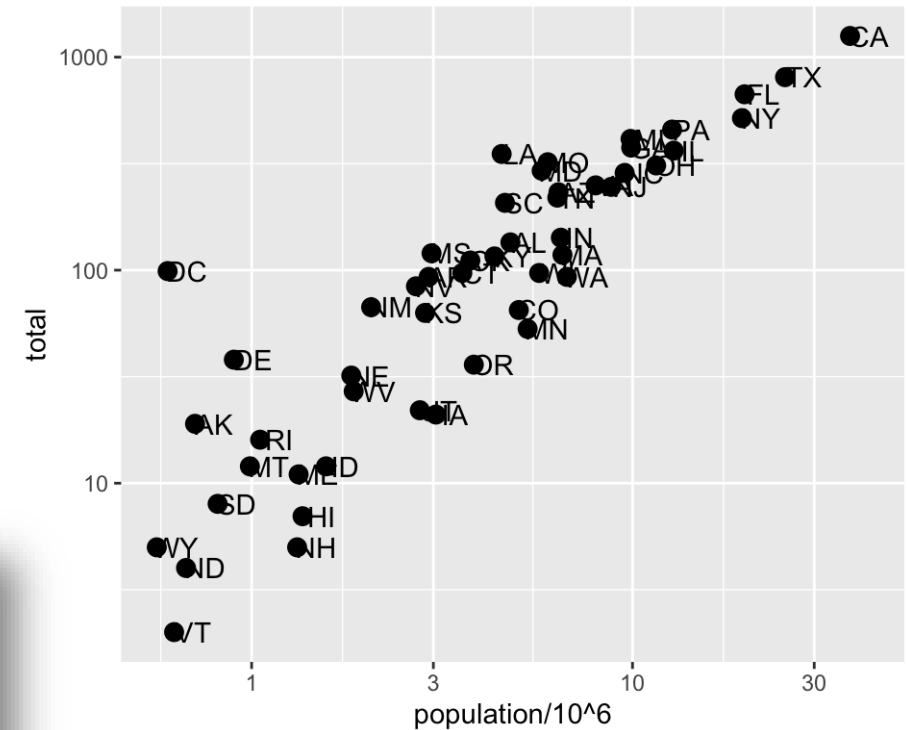


Scaling axes

Our desired scales are in log-scale. This is not the default, so this change needs to be added through another layer. The `scale_x_continuous` and `scale_y_continuous` function lets us control the behavior of scaling axes.

Also try to use `scale_x_log10` and `scale_y_log10`

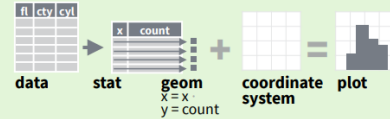
```
> ggplot(murders, aes(population/10^6, total, label = abb)) +  
  geom_point(size = 3) +  
  geom_text(nudge_x = 0.05) +  
  scale_x_continuous(trans = "log10") +  
  scale_y_continuous(trans = "log10")
```



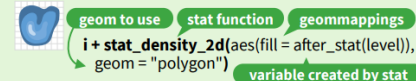
Stats

An alternative way to build a layer.

A stat builds new variables to plot (e.g., count, prop).



Visualize a stat by changing the default stat of a geom function, `geom_bar(stat="count")` or by using a stat function, `stat_count(geom="bar")`, which calls a default geom to make a layer (equivalent to a geom function). Use `after_stat(name)` syntax to map the stat variable `name` to an aesthetic.



```
c + stat_bin(binwidth = 1, boundary = 10)
x, y | count, ncount, density, ndensity
```

```
c + stat_count(width = 1) x, y | count, prop
```

```
c + stat_density(adjust = 1, kernel = "gaussian")
x, y | count, density, scaled
```

```
e + stat_bin_2d(bins = 30, drop = T)
x, y, fill | count, density
```

```
e + stat_bin_hex(bins = 30) x, y, fill | count, density
```

```
e + stat_density_2d(contour = TRUE, n = 100)
x, y, color, size | level
```

```
e + stat_ellipse(level = 0.95, segments = 51, type = "t")
```

```
l + stat_contour(aes(z = z)) x, y, z, order | level
```

```
l + stat_summary_hex(aes(z = z), bins = 30, fun = max)
x, y, z, fill | value
```

```
l + stat_summary_2d(aes(z = z), bins = 30, fun = mean)
x, y, z, fill | value
```

```
f + stat_boxplot(coef = 1.5)
x, y | lower, middle, upper, width, ymin, ymax
```

```
f + stat_ydensity(kernel = "gaussian", scale = "area") x, y |
density, scaled, count, n, violinwidth, width
```

```
e + stat_ecdf(n = 40) x, y | x, y
```

```
e + stat_quantile(quantiles = c(0.1, 0.9),
formula = y ~ log(x), method = "rq") x, y | quantile
```

```
e + stat_smooth(method = "lm", formula = y ~ x, se = T,
level = 0.95) x, y | se, x, y, ymin, ymax
```

```
ggplot() + xlim(-5, 5) + stat_function(fun = dnorm,
n = 20, geom = "point") x | x, y
```

```
ggplot() + stat_qq(aes(sample = 1:100))
x, y, sample | sample, theoretical
```

```
e + stat_sum() x, y, size | n, prop
```

```
e + stat_summary(fun.data = "mean_cl_boot")
```

```
h + stat_summary_bin(fun = "mean", geom = "bar")
```

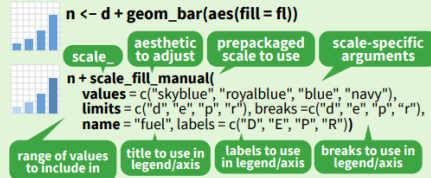
```
e + stat_identity()
```

```
e + stat_unique()
```

Scales

Override defaults with `scales` package.

Scales map data values to the visual values of an aesthetic. To change a mapping, add a new scale.



GENERAL PURPOSE SCALES

Use with most aesthetics

scale_*_continuous() - Map cont' values to visual ones.

scale_*_discrete() - Map discrete values to visual ones.

scale_*_binned() - Map continuous values to discrete bins.

scale_*_identity() - Use data values as visual ones.

scale_*_manual(values = c()) - Map discrete values to manually chosen visual ones.

scale_*_date(date_labels = "%m/%d"),
date_breaks = "2 weeks" - Treat data values as dates.

scale_*_datetime() - Treat data values as date times.
Same as `scale_*_date()`. See ?strptime for label formats.

X & Y LOCATION SCALES

Use with x or y aesthetics (x shown here)

scale_x_log10() - Plot x on log10 scale.

scale_x_reverse() - Reverse the direction of the x axis.

scale_x_sqrt() - Plot x on square root scale.

COLOR AND FILL SCALES (DISCRETE)

n + scale_fill_brewer(palette = "Blues")
For palette choices:
RColorBrewer::display.brewer.all()

n + scale_fill_grey(start = 0.2, end = 0.8, na.value = "red")

COLOR AND FILL SCALES (CONTINUOUS)

o <- c + geom_dotplot(aes(fill = x))

o + scale_fill_distiller(palette = "Blues")

o + scale_fill_gradient(low = "red", high = "yellow")

o + scale_fill_gradient2(low = "red", high = "blue", mid = "white", midpoint = 25)

o + scale_fill_gradientn(colors = topo.colors(6))
Also: `rainbow()`, `heat.colors()`, `terrain.colors()`,
`cm.colors()`, `RColorBrewer::brewer.pal()`

SHAPE AND SIZE SCALES

p <- e + geom_point(aes(shape = fl, size = cyl))

p + scale_shape() + scale_size()
p + scale_shape_manual(values = c(3:7))

p + scale_radius(range = c(1,6))
p + scale_size_area(max_size = 6)

Coordinate Systems

```
r <- d + geom_bar()
```

r + coord_cartesian(xlim = c(0, 5)) - xlim, ylim
The default cartesian coordinate system.

r + coord_fixed(ratio = 1/2)
ratio, xlim, ylim - Cartesian coordinates with fixed aspect ratio between x and y units.

r + coord_flip()
Flip cartesian coordinates by switching x and y aesthetic mappings.

r + coord_polar(theta = "x", direction = 1)
theta, start, direction - Polar coordinates.

r + coord_trans(y = "sqrt") - x, y, xlim, ylim
Transformed cartesian coordinates. Set xtrans and ytrans to the name of a window function.

π + coord_sf() - xlim, ylim, crs. Ensures all layers use a common Coordinate Reference System.

Position Adjustments

Position adjustments determine how to arrange geoms that would otherwise occupy the same space.

```
s <- ggplot(mpg, aes(fl, fill = drv))
```

s + geom_bar(position = "dodge")
Arrange elements side by side.

s + geom_bar(position = "fill")
Stack elements on top of one another, normalize height.

e + geom_point(position = "jitter")
Add random noise to X and Y position of each element to avoid overplotting.

e + geom_label(position = "nudge")
Nudge labels away from points.

s + geom_bar(position = "stack")
Stack elements on top of one another.

Each position adjustment can be recast as a function with manual **width** and **height** arguments:

```
s + geom_bar(position = position_dodge(width = 1))
```

Themes

r + theme_bw()
White background with grid lines.

r + theme_gray()
Grey background (default theme).

r + theme_dark()
Dark for contrast.

r + theme_classic()
r + theme_light()

r + theme_linedraw()
Minimal theme.

r + theme_void()
Empty theme.

r + theme() Customize aspects of the theme such as axis, legend, panel, and facet properties.
`r + labs(title = "Title") + theme(plot.title.position = "plot")`
`r + theme(panel.background = element_rect(fill = "blue"))`

Faceting

Facets divide a plot into subplots based on the values of one or more discrete variables.

```
t <- ggplot(mpg, aes(cty, hwy)) + geom_point()
```

t + facet_grid(. ~ fl)
Facet into columns based on fl.

t + facet_grid(year ~ .)
Facet into rows based on year.

t + facet_grid(year ~ fl)
Facet into both rows and columns.

t + facet_wrap(~ fl)
Wrap facets into a rectangular layout.

Set **scales** to let axis limits vary across facets.

t + facet_grid(drv ~ fl, scales = "free")
x and y axis limits adjust to individual facets:
"free_x" - x axis limits adjust
"free_y" - y axis limits adjust

Set **labeller** to adjust facet label:

```
t + facet_grid(. ~ fl, labeller = label_both)
```

fl: c	fl: d	fl: e	fl: p	fl: r
α^c	α^d	α^e	α^p	α^r

```
t + facet_grid(fl ~ ., labeller = label_bquote(alpha ^ (.fl)))
```

Labels and Legends

Use **labs()** to label the elements of your plot.

t + labs(x = "New x axis label", y = "New y axis label", title = "Add a title above the plot", subtitle = "Add a subtitle below title", caption = "Add a caption below plot", alt = "Add alt text to the plot", <AES> = "New <AES> legend title")

t + annotate(geom = "text", x = 8, y = 9, label = "A")
Places a geom with manually selected aesthetics.

p + guides(x = guide_axis(n.dodge = 2)) Avoid crowded or overlapping labels with `guide_axis(n.dodge or angle)`.

n + guides(fill = "none") Set legend type for each aesthetic: colorbar, legend, or none (no legend).

n + theme(legend.position = "bottom")
Place legend at "bottom", "top", "left", or "right".

n + scale_fill_discrete(name = "Title", labels = c("A", "B", "C", "D", "E"))
Set legend title and labels with a scale function.

Zooming

Without clipping (preferred):

```
t + coord_cartesian(xlim = c(0, 100), ylim = c(10, 20))
```

With clipping (removes unseen data points):

```
t + xlim(0, 100) + ylim(10, 20)
```

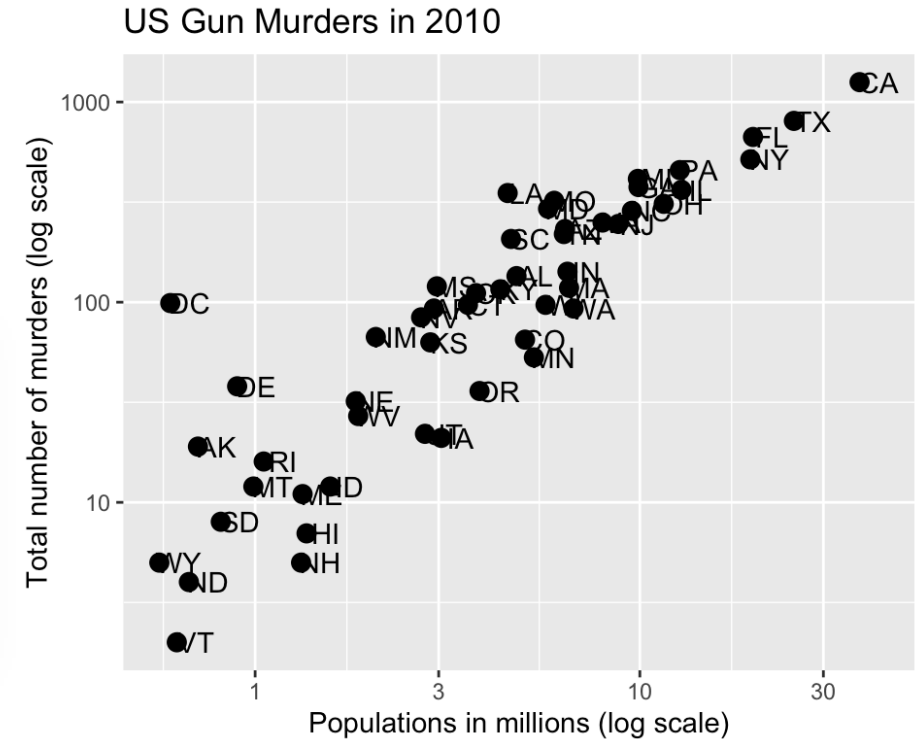
```
t + scale_x_continuous(limits = c(0, 100)) +
scale_y_continuous(limits = c(0, 100))
```

Labels and titles

Labels are just variable names by default

Let's come up with some more meaningful names

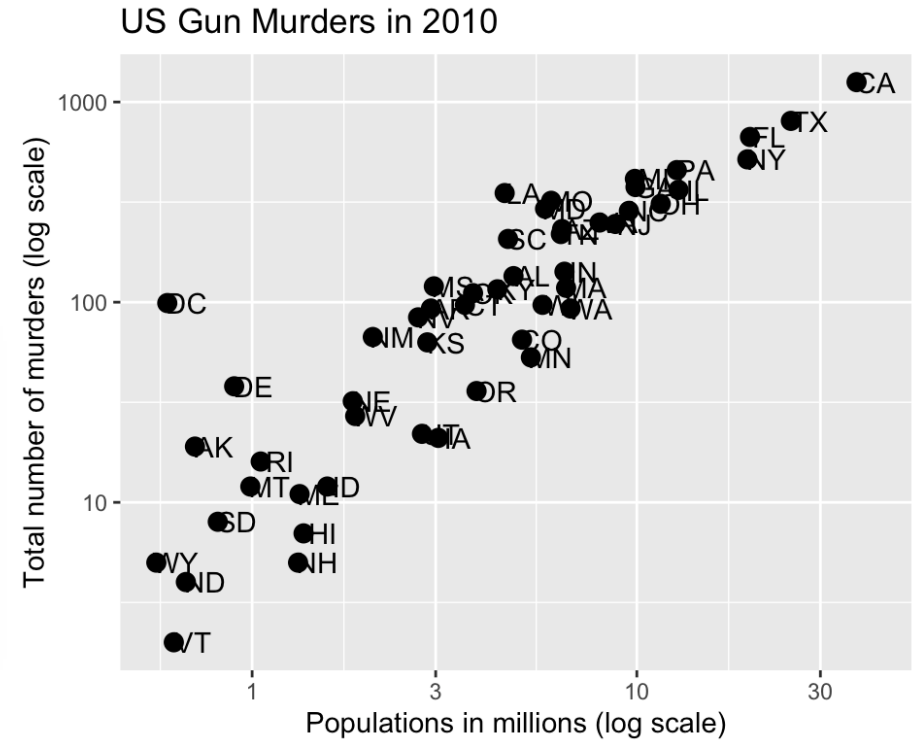
```
> ggplot(murders, aes(population/10^6, total, label = abb)) +  
+ geom_point(size = 3) +  
+ geom_text(nudge_x = 0.05) +  
+ scale_x_log10() +  
+ scale_y_log10() +  
+ labs(x = "Populations in millions (log scale)",  
+       y = "Total number of murders (log scale)",  
+       title = "US Gun Murders in 2010")
```



Labels and titles

Another alternative to make it more “layered”

```
> ggplot(murders, aes(population/10^6, total, label = abb)) +  
+ geom_point(size = 3) +  
+ geom_text(nudge_x = 0.05) +  
+ scale_x_log10() +  
+ scale_y_log10() +  
+ xlab("Populations in millions (log scale)") +  
+ ylab("Total number of murders (log scale)") +  
+ ggtitle("US Gun Murders in 2010")
```

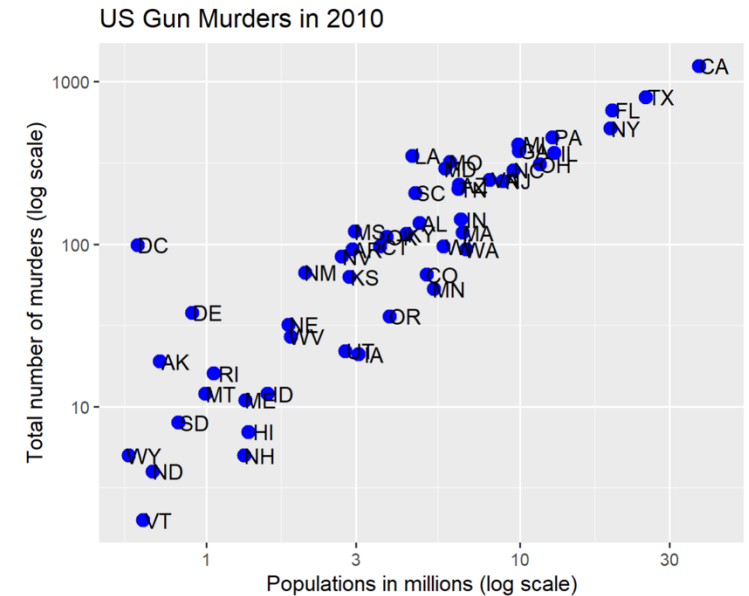


Assigning colors to data points

We can change the color of the points using the `color` argument of `geom_point`

- Note that this setting is not data point-dependent, meaning all points will be of the same color

```
> murders |> ggplot(aes(population/10^6, total, label = abb)) +  
+   geom_point(size = 3, color = "blue") +  
+   geom_text(nudge_x = 0.05) +  
+   scale_x_log10() +  
+   scale_y_log10() +  
+   xlab("Populations in millions (log scale)") +  
+   ylab("Total number of murders (log scale)") +  
+   ggtitle("US Gun Murders in 2010")
```

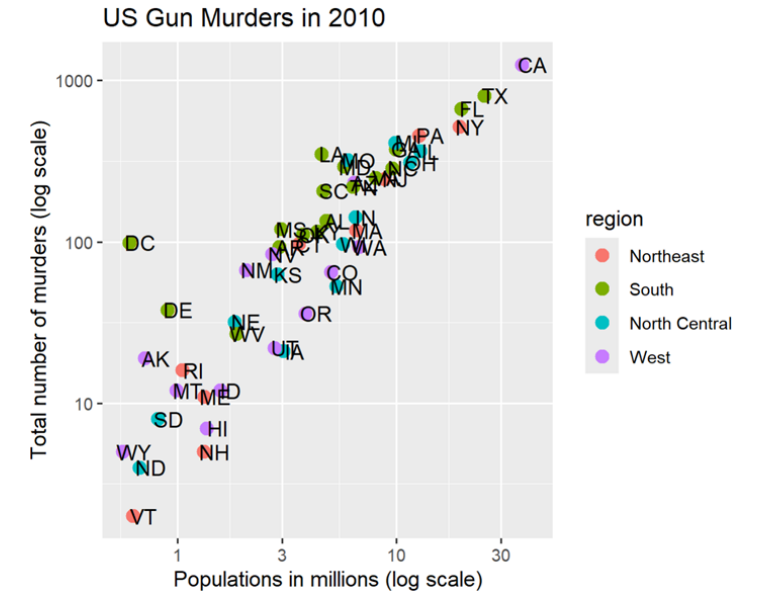


Assigning colors to data points

An alternative is to link color to a data-dependent variable. This will need to be done in `aes`.

- Here colors of data points will be dependent on the specified variable “region”.

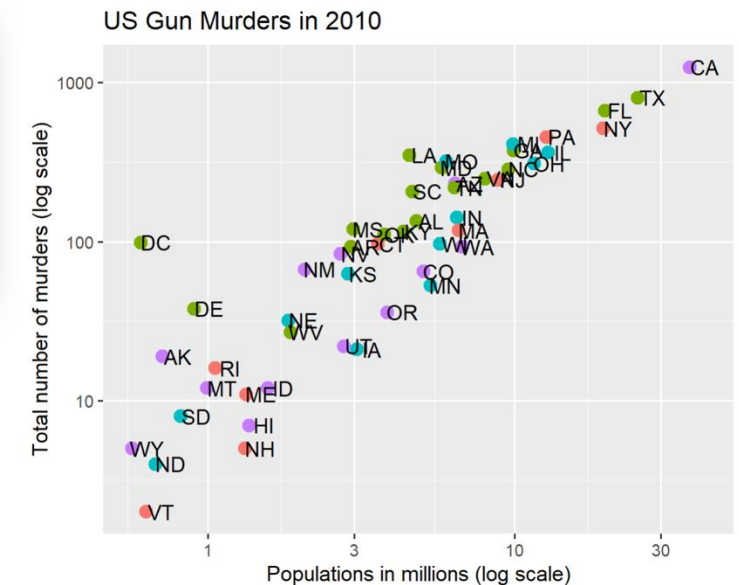
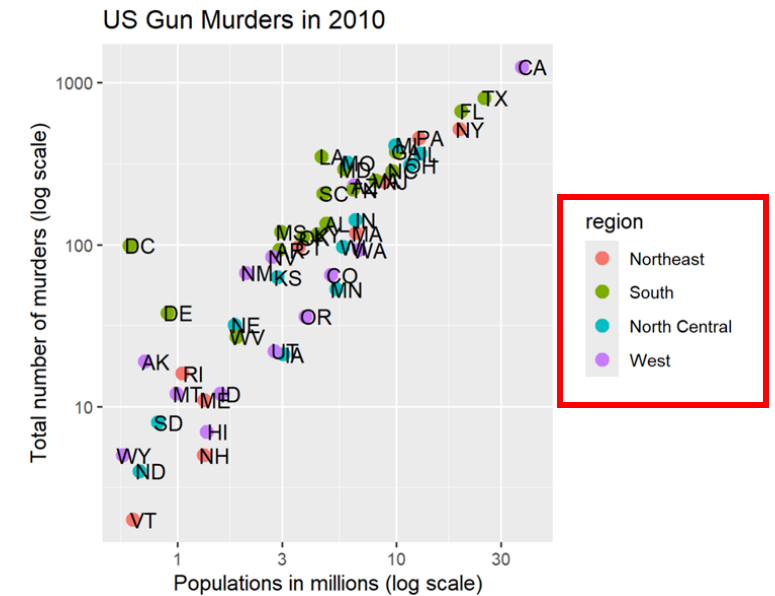
```
> murders |> ggplot(aes(population/10^6, total, label = abb)) +  
+   geom_point(aes(color=region) size = 3) +  
+   geom_text(nudge_x = 0.05) +  
+   scale_x_log10() +  
+   scale_y_log10() +  
+   xlab("Populations in millions (log scale)") +  
+   ylab("Total number of murders (log scale)") +  
+   ggtitle("US Gun Murders in 2010")
```



Assigning colors to data points

- The `color` argument in `local aes` is used in conjunction with global `aes` mappings (`x`, `y`, `label`)
- Set `show.legend=FALSE` to remove the legend

```
> murders |> ggplot(aes(population/10^6, total, label = abb)) +  
+   geom_point(aes(color=region), size = 3, show.legend=FALSE) +  
+   geom_text(nudge_x = 0.05) +  
+   scale_x_log10() +  
+   scale_y_log10() +  
+   xlab("Populations in millions (log scale)") +  
+   ylab("Total number of murders (log scale)") +  
+   ggtitle("US Gun Murders in 2010")
```



Adding the global trend

How does each state compare to national average?

How to calculate national average?

```
> r <- murders |>  
+   summarize(rate = sum(total) / sum(population) * 10^6) |>  
+   pull(rate)  
> r  
[1] 30.34555
```

Which line represents national average?

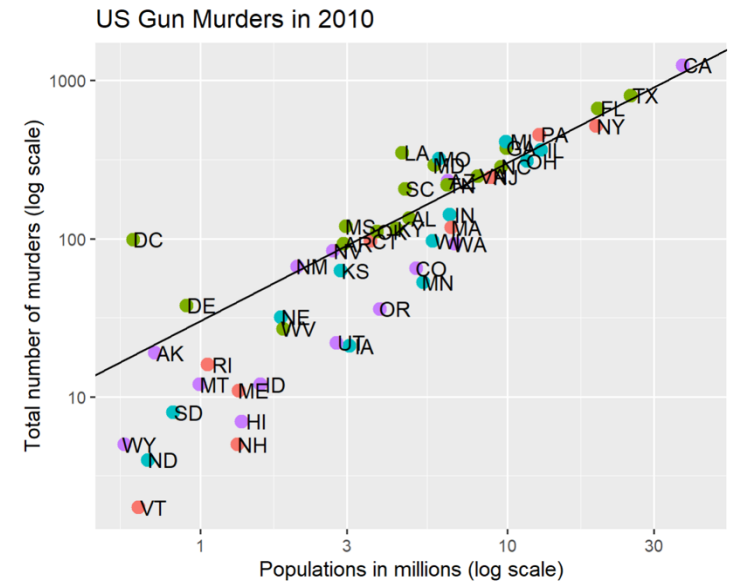
$$y = r * x$$

y — total number of murders

x — population in millions

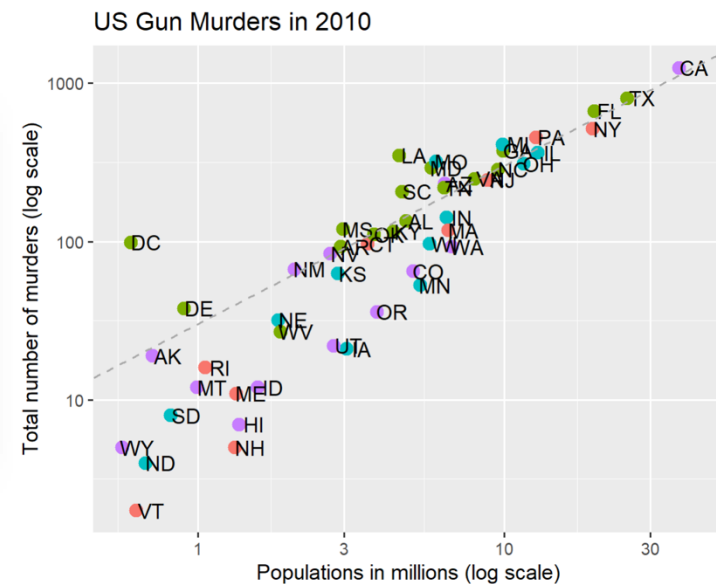
And on log scale: $\log(y) = \log(x) + \log(r)$

```
> murders |> ggplot(aes(population/10^6, total, label = abb)) +  
+   geom_point(aes(color=region), size = 3, show.legend=FALSE) +  
+   geom_text(nudge_x = 0.05) +  
+   scale_x_log10() +  
+   scale_y_log10() +  
+   geom_abline(slope=1, intercept=log(r, base=10)) +  
+   xlab("Populations in millions (log scale)") +  
+   ylab("Total number of murders (log scale)") +  
+   ggtitle("US Gun Murders in 2010")
```



Use different arguments to specify the styles of `geom_abline`

```
> murders |> ggplot(aes(population/10^6, total, label = abb)) +
+   geom_point(aes(color=region), size = 3, show.legend=FALSE) +
+   geom_text(nudge_x = 0.05) +
+   scale_x_log10() +
+   scale_y_log10() +
+   geom_abline(slope=1, intercept=log(r, base=10),
+               lty="dashed", color="darkgrey") +
+   xlab("Populations in millions (log scale)") +
+   ylab("Total number of murders (log scale)") +
+   ggtitle("US Gun Murders in 2010")
```



You can use:

- Line styles (`lty`)
- Colors (`color`)
- Line widths (`size`)

6. 'twodash'	- - - - -
5. 'longdash'	_ _ _ _ _
4. 'dotdash'	- . - . - .
3. 'dotted'	
2. 'dashed'	- - - - -
1. 'solid'	_____
0. 'blank'	

Where to ask if you have unmet needs?

Here are my codes for plotting the murders dataset in R:

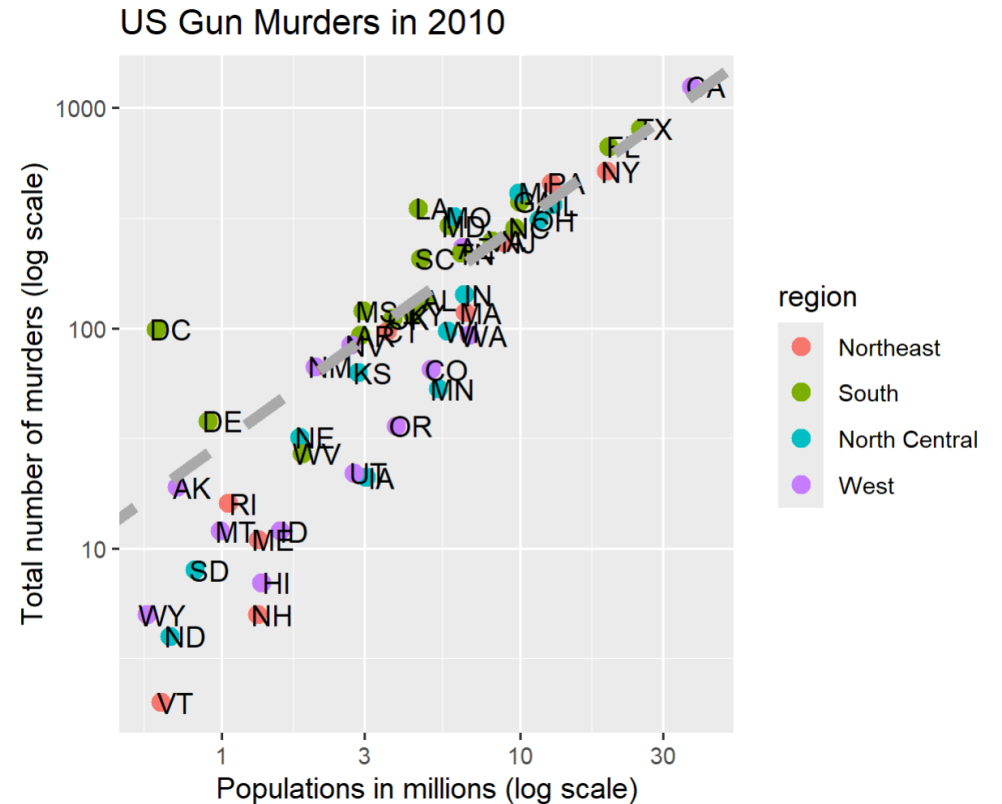
```
####
```

```
r <- murders |>
  summarize(rate = sum(total) / sum(population) * 10^6) |>
  pull(rate)
```

```
murders |> ggplot(aes(population/10^6, total, label = abb)) +
  geom_point(aes(color=region), size = 3) +
  geom_text(nudge_x = 0.05) +
  scale_x_log10() +
  scale_y_log10() +
  geom_abline(slope=1, intercept=log(r, base=10),
             lty=2, color="darkgrey", size=2) +
  xlab("Populations in millions (log scale)") +
  ylab("Total number of murders (log scale)") +
  ggtitle("US Gun Murders in 2010")
####
```

How can I:

1. Rename the title of the legend section (from "region" to "Region")
2. Make the trend line appear behind the scatter plot dots



Where to ask if you have unmet needs?

Got it — you want to:

1. Rename the legend title (from "region" to "Region")
2. Move the trend line so it's plotted *behind* the scatter plot dots.

Here's how you can do it step-by-step:

1. Rename the legend title

Since your legend is coming from the `color=region` mapping in `geom_point()`, you can use:

```
labs(color = "Region")
```

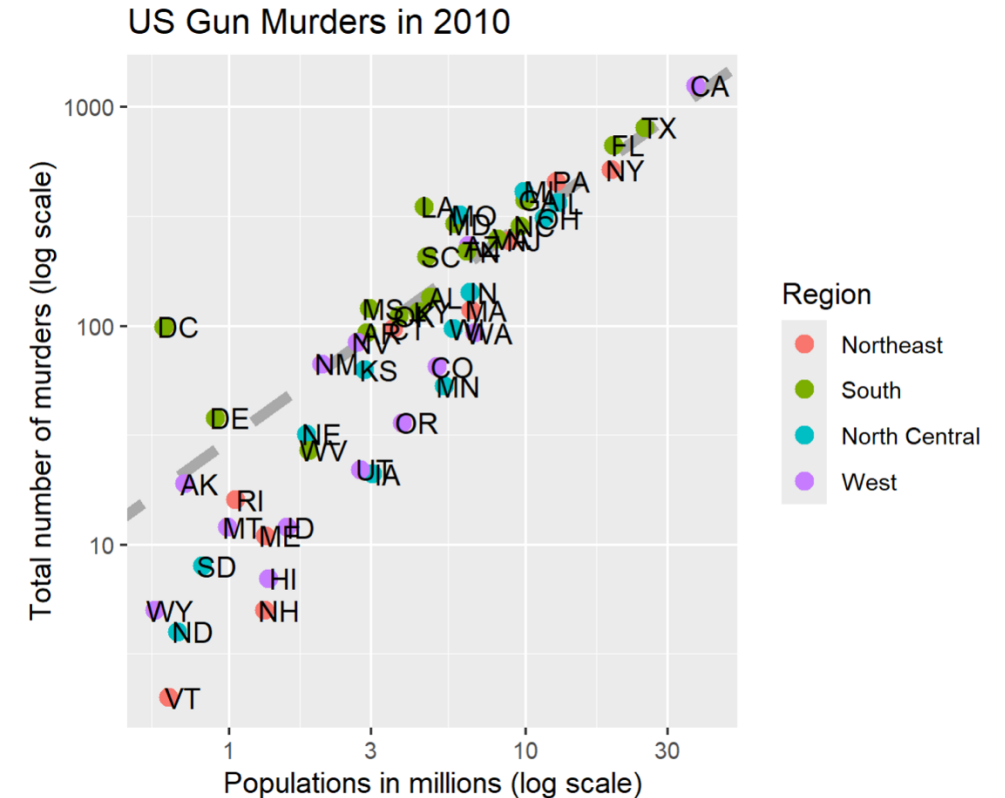
This will replace "region" with "Region" in the legend.

2. Put the trend line behind the points

In `ggplot2`, the order in which you add geoms determines their plotting order — earlier geoms are drawn first (i.e., in the background).

Right now, your `geom_abline()` is after the points, so it's drawn *on top*. To move it behind, put `geom_abline()` before `geom_point()`:

Try google it, GPT it, ...

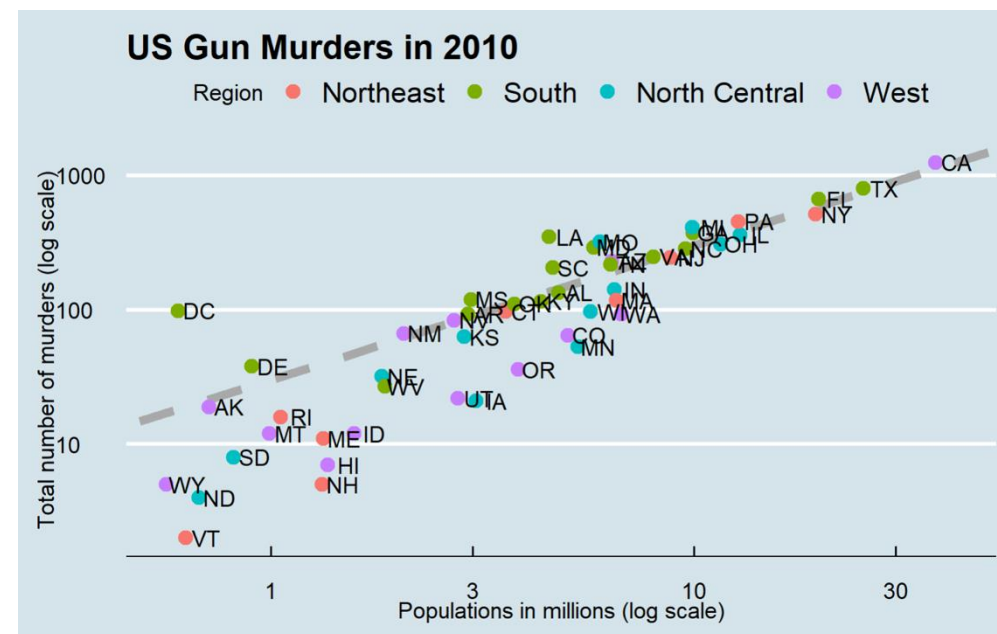


Beautify your plot

The power of `ggplot2` is augmented further due to the availability of add-on packages

The style of the graph can be changed using `ggthemes` functions.

```
> library(ggthemes)
> murders |> ggplot(aes(population/10^6, total, label = abb)) +
+   geom_abline(slope=1, intercept=log(r, base=10),
+             lty=2, color="darkgrey", size=2) +
+   geom_point(aes(color=region), size = 3) +
+   geom_text(nudge_x = 0.05) +
+   scale_x_log10() +
+   scale_y_log10() +
+   xlab("Populations in millions (log scale)") +
+   ylab("Total number of murders (log scale)") +
+   ggtitle("US Gun Murders in 2010") +
+   labs(color = "Region") +
+   theme_economist()
```

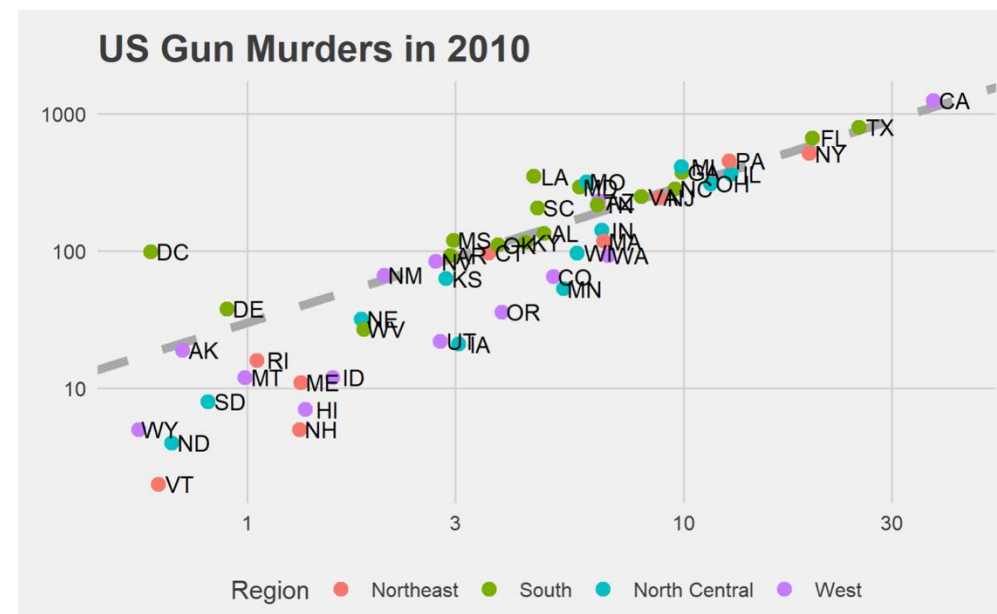


Beautify your plot

The power of `ggplot2` is augmented further due to the availability of add-on packages

The style of the graph can be changed using `ggthemes` functions.

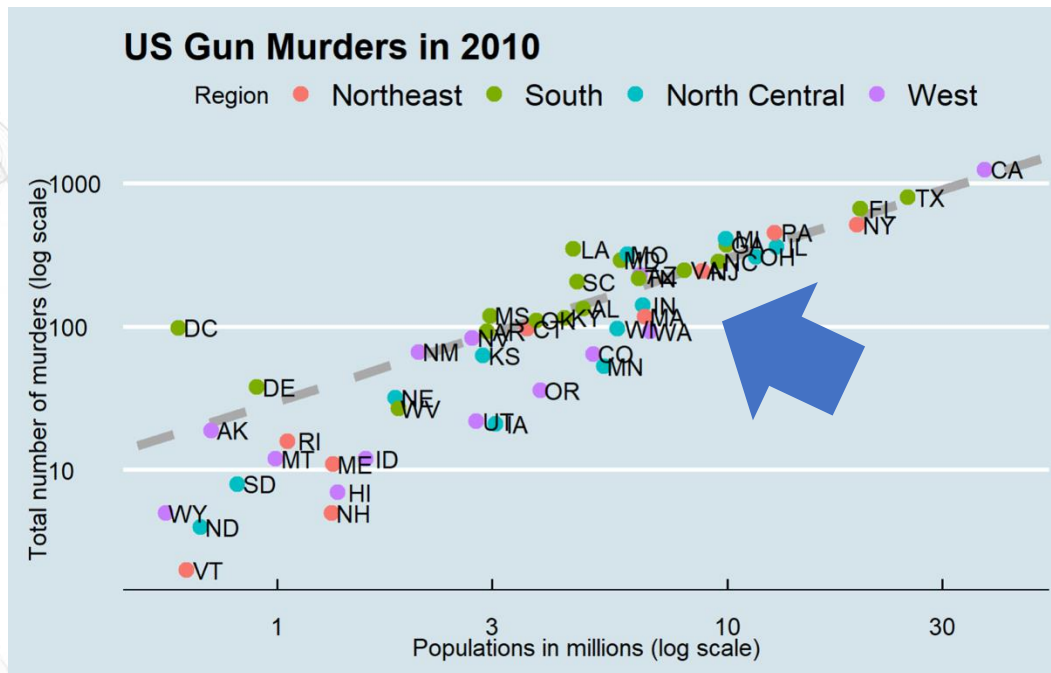
```
> library(ggthemes)
> murders |> ggplot(aes(population/10^6, total, label = abb)) +
+   geom_abline(slope=1, intercept=log(r, base=10),
+               lty=2, color="darkgrey", size=2) +
+   geom_point(aes(color=region), size = 3) +
+   geom_text(nudge_x = 0.05) +
+   scale_x_log10() +
+   scale_y_log10() +
+   xlab("Populations in millions (log scale)") +
+   ylab("Total number of murders (log scale)") +
+   ggtitle("US Gun Murders in 2010") +
+   labs(color = "Region") +
+   theme_fivethirtyeight()
```



A complete list of themes in `ggthemes`:

<https://yutannihilation.github.io/allYourFigureAreBelongToUs/ggthemes/>

When there is a will, there is a way,
in R at least



Question:

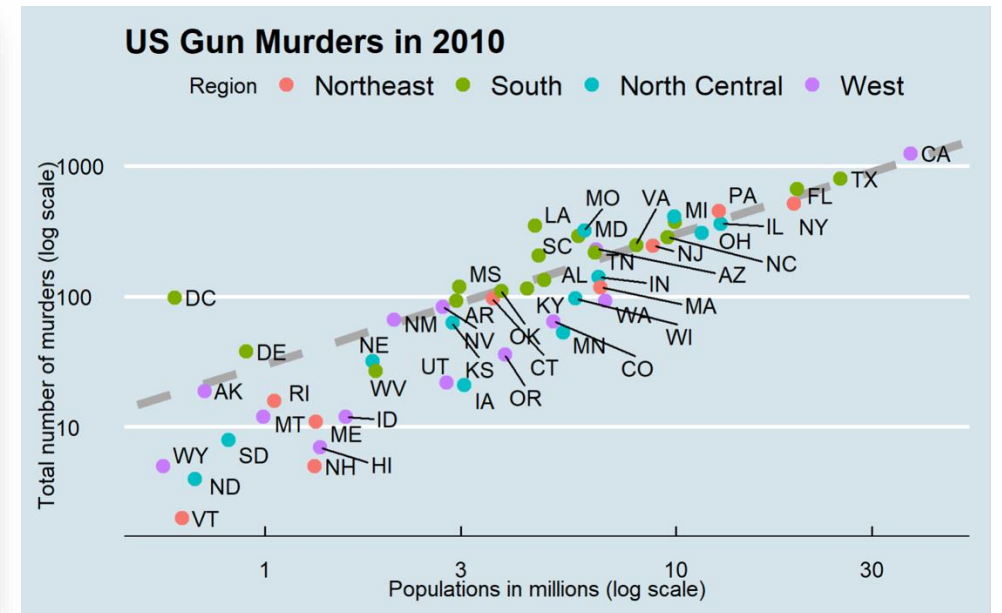
How to rearrange the labels
in R so they don't overlap?

How to rearrange the labels in R so they don't overlap?

If you want to avoid label overlap in your **murders** scatter plot, you can replace `geom_text()` with `geom_text_repel()` from the `ggrepel` package.

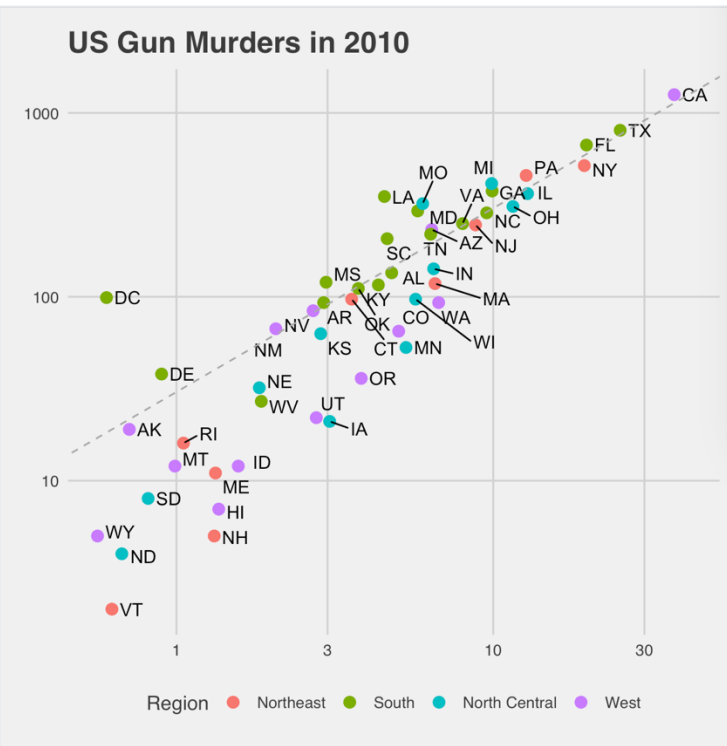
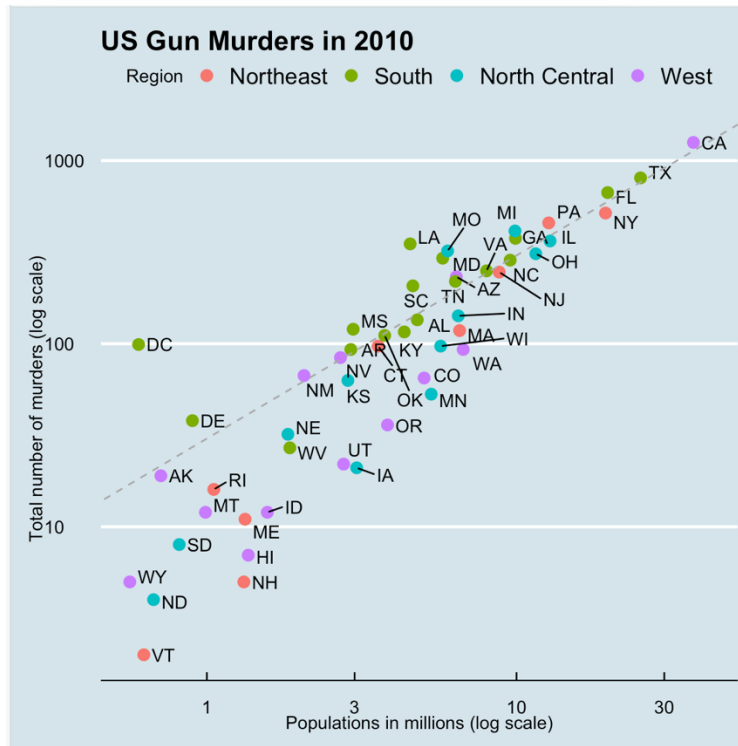
Putting it all together

```
1 library(tidyverse)
2 library(ggthemes)
3 library(ggrepel)
4 library(dslabs)
5
6 data(murders)
7
8 r <- murders |>
9   summarize(rate = sum(total) / sum(population) * 10^6) |>
10  pull(rate)
11
12 murders |> ggplot(aes(population/10^6, total, label = abb)) +
13   geom_abline(slope=1, intercept=log(r, base=10),
14             lty=2, color="darkgrey", size=2) +
15   geom_point(aes(color=region), size = 3) +
16   geom_text_repel(nudge_x = 0.05) +
17   scale_x_log10() +
18   scale_y_log10() +
19   xlab("Populations in millions (log scale)") +
20   ylab("Total number of murders (log scale)") +
21   ggtitle("US Gun Murders in 2010") +
22   labs(color = "Region") +
23   theme_economist()
```



Grids of plots

```
library(gridExtra)
p1 <- # plot 1 code
p2 <- # plot 2 code
grid.arrange(p1, p2, ncol = 2)
```



Takeaway

- `ggplot2` used this layered approach to construct beautiful graphs step by step.
- You don't need to remember all the individual components. Use the cheatsheet/Google/GPT to help you.
- Knowing what/why you are plotting is more important than knowing how to plot.

How I see the world after ggplot2





In this lecture

- Data visualization and its importance
- ggplot2 basics
- **Visualize data distributions**



Data distributions

Data distribution refers to the way data is spread out or arranged across a range of values.

Q: how much do people earn in Hong Kong?

A: The median salary is HKD 83.6 per hour (~14,500 HKD per month) for employee in Hong Kong in 2024.

One number is often not enough. What else do we want to learn?

- Central tendencies: mean, median, etc.
- Spread/dispersion: variance, standard deviation, IQR, etc.
- Patterns: Symmetric? Skewed? Bi-modal? Uniform? Heavy-tailed?

What is a better way to communicate all these information?

Visualize data distributions



What else can you learn from the plot on the left?

The most straightforward way to understand a list of values is to look at its distribution.

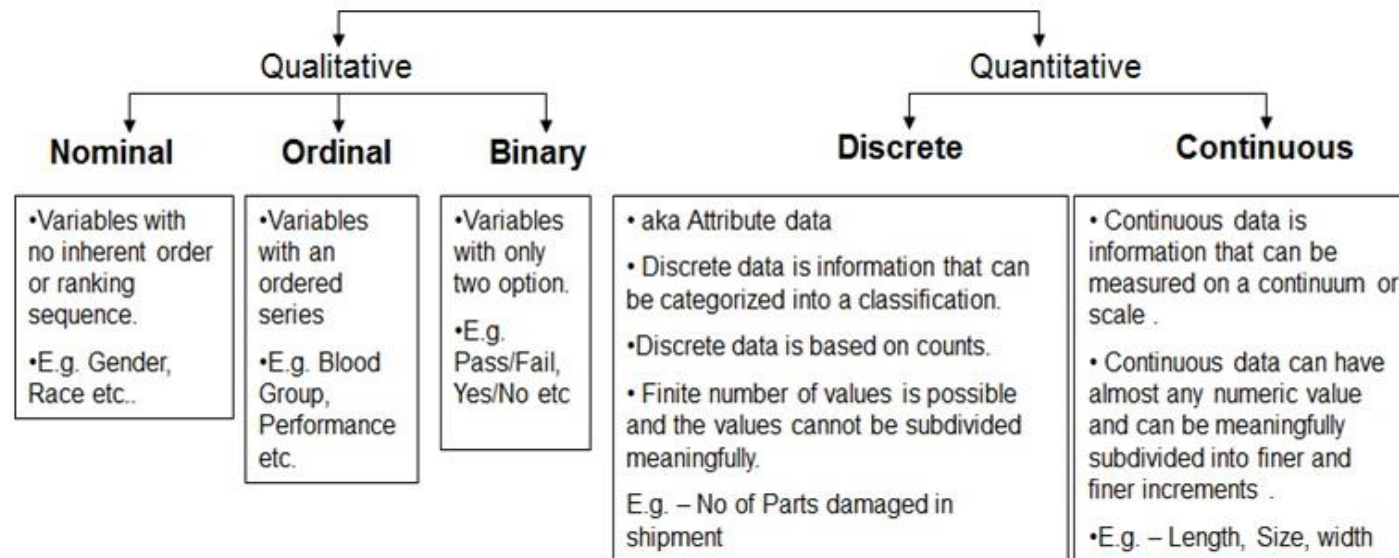
We will go over several data visualization techniques that can effectively show and communicate the distributions of data.

From: Census and Statistics Department

<https://www.censtatd.gov.hk/en/wbr.html?ecode=B10500142024AN24&scode=210>

Variables and variable types

- The two main variable types are categorical/numeric
 - A categorical variable can be ordinal or nominal
 - Ordinal: {very bad, bad, neutral, good, very good}
 - Nominal: {Northeast, South, North Central, West}
 - A numerical variable can be discrete or continuous
 - Discrete: e.g., population size
 - Continuous: e.g., heights



Variables and variable types

We will use both the `murders` and `heights` datasets for following examples.

```
> library(tidyverse)
> library(dslabs)
> data(heights)
> heights <- mutate(heights, height = height*2.54)
```

Skip this if you are comfortable working with
U.S. customary system (inches, etc.)

```
> str(heights)
'data.frame':  1050 obs. of  2 variables:
 $ sex    : Factor w/ 2 levels "Female","Male": 2 2 2 2 2 1 1 1 1 2 ...
 $ height: num  190 178 173 188 155 ...
```

R: Self-Reported Heights ▾ Find in Topic

heights {dslabs}

Self-Reported Heights

Description

Self-reported heights in inches for males and females.

Usage

```
data(heights)
```

Format

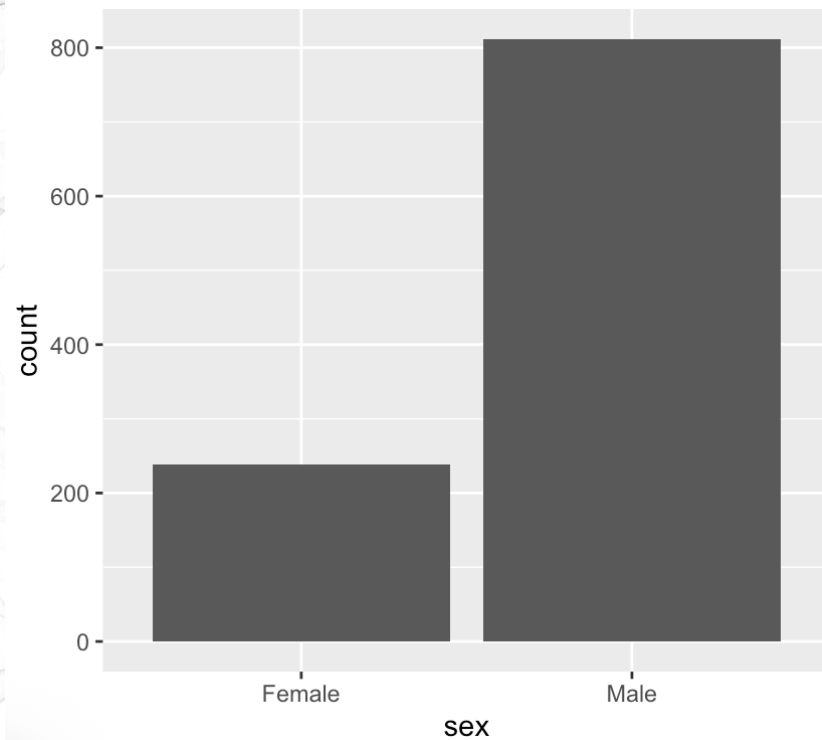
An object of class "data.frame".

Details

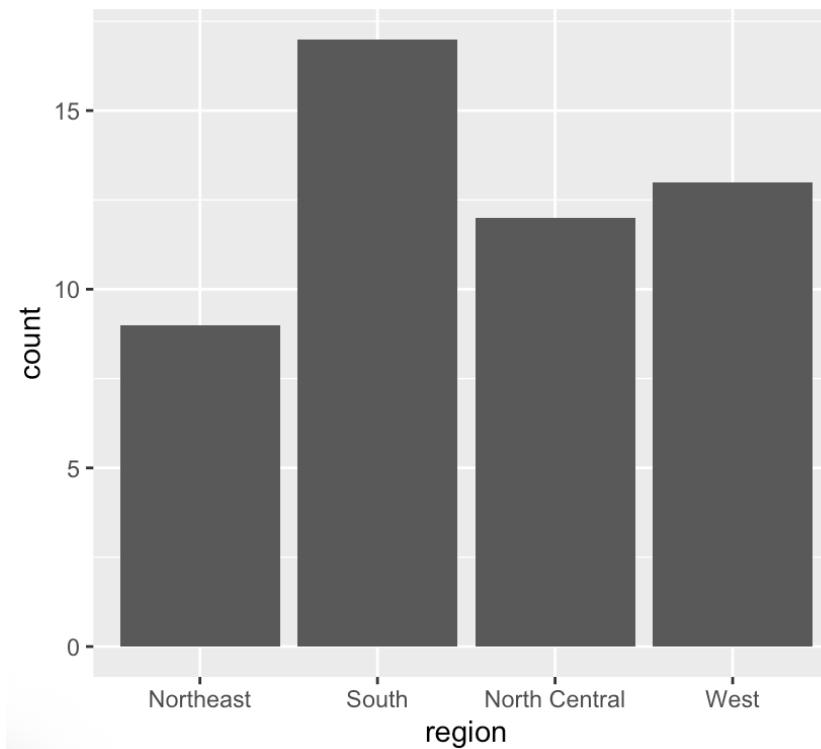
- sex. Male or Female.
- height. Height in inches.

Count of categorical data using **bar plot**: `geom_bar`

```
> heights |> ggplot(aes(sex)) + geom_bar()
```



```
> murders |> ggplot(aes(region)) + geom_bar()
```



Proportion of categorical data

With categorical data, the proportion of each unique category describes the distribution

```
> heights |>  
+ count(sex) |>  
+ mutate(proportion = n/sum(n)) |>  
+ ggplot(aes(sex, proportion)) + geom_col()
```

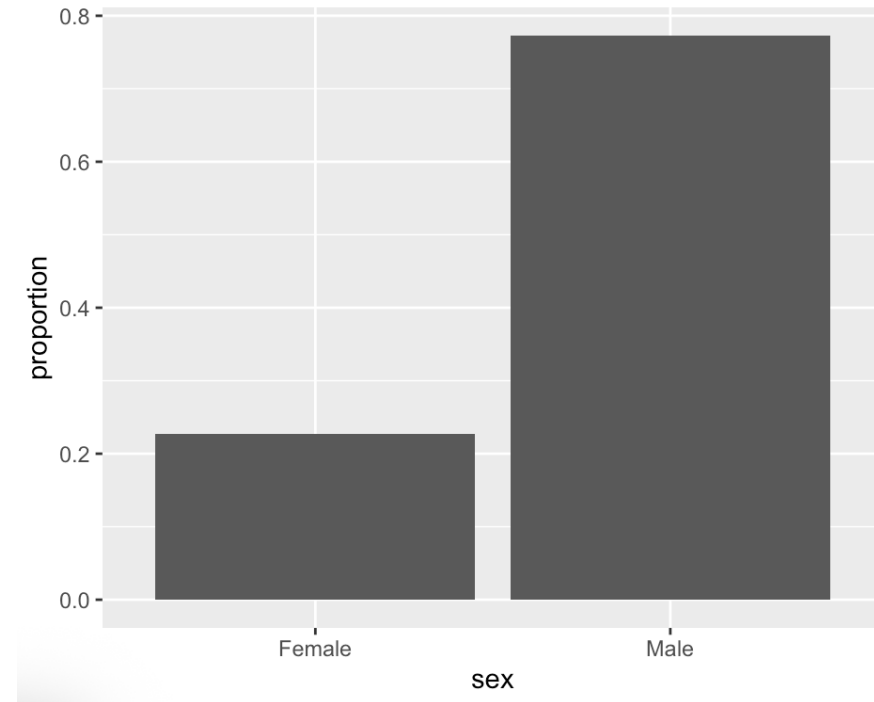
`count` yield a new column `n`, which stores the counts of different entries.

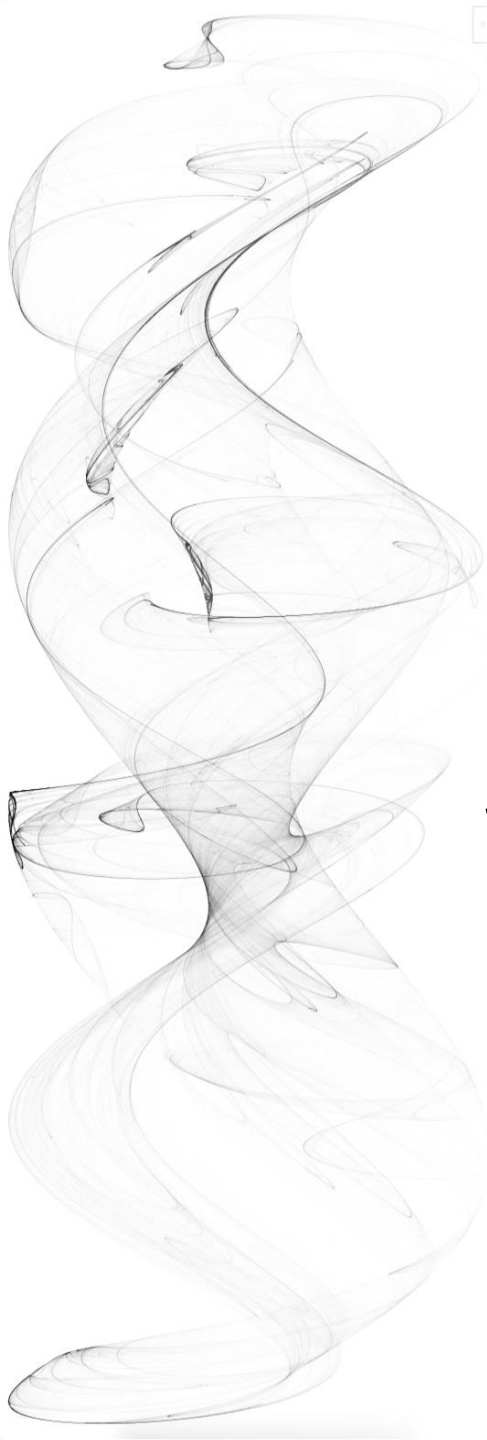
We use `geom_col` here, which requires `x` and `y` mappings.

`geom_bar` only requires `x` mapping, and it calculates count of categorical variable automatically.

See also:

<https://stackoverflow.com/a/59009108>





What has been plotted?

In a bar plot (`geom_bar` or `geom_col`):

- One axis is categorical: male vs female, etc.
- The other axis is numerical: count/proportion of the categorical variable.

What if the variable of interest is numerical/continuous?

Histogram

- **When data is not categorical, reporting the frequency of each entry, as we did for categorical data using barplot, is not an effective summary. Why?**
- We use histogram instead: histogram divides the span of data into non-overlapping bins. Then, for each bin, we count the number of values that fall in that interval.

```
> heights |> count(height)
```

	height	n
1	127.0000	2
2	129.5400	1
3	132.0800	2
4	134.6200	1
5	136.5758	1
6	137.1600	1
7	139.7000	1
8	147.3200	1
9	149.8600	6
10	150.0000	1
11	150.0000	2
12	152.0000	1
13	152.4000	17
14	154.9400	12
15	155.7528	1
16	157.0000	2
17	157.4800	19
18	158.0000	3
19	158.4960	1
20	158.7500	2
21	159.0000	1

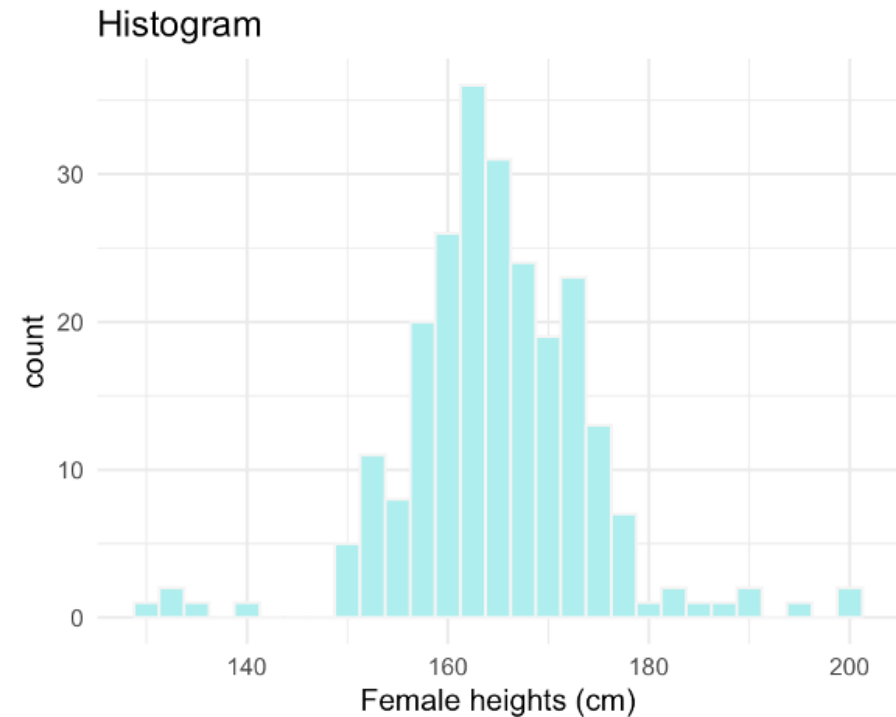
60 inches (5 feet)

61 inches

Histogram

Here is the histogram for the height data splitting the range of values into 2.5cm (~1 inch) intervals:

```
heights |>
  filter(sex == "Female") |>
  ggplot(aes(height)) +
  geom_histogram(binwidth=2.5,
                 fill='paleturquoise',
                 col='whitesmoke') +
  xlab("Female heights (cm)") +
  ggtitle("Histogram") +
  theme_minimal()
```



A histogram is similar in concept to a bar plot, but it differs in that the x-axis is numerical, not categorical

Pick your vibe

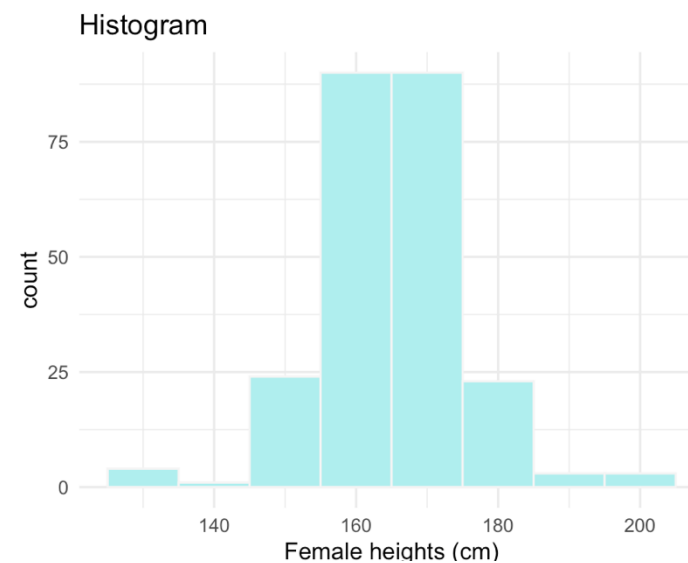
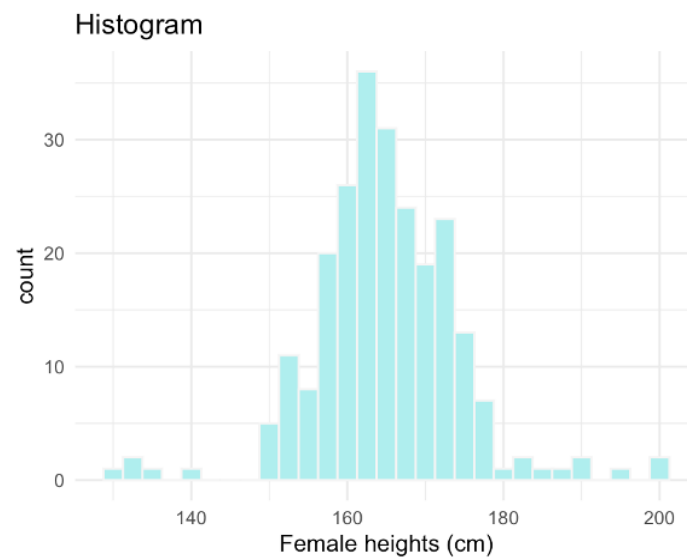
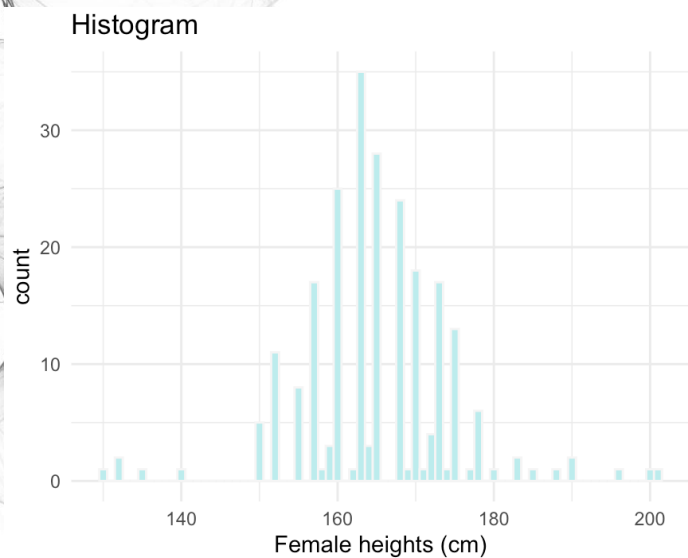
<https://www.datanovia.com/en/blog/awesome-list-of-657-r-color-names/>



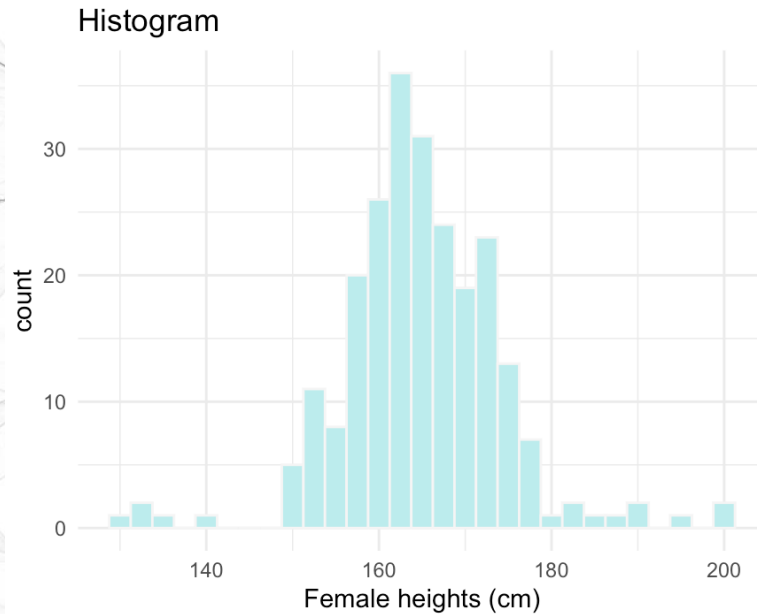
What's the best binwidth?

- (from left to right)

`binwidth = 1`, `binwidth = 2.5`, `binwidth = 10`



Histogram



What can we learn immediately from the histogram?

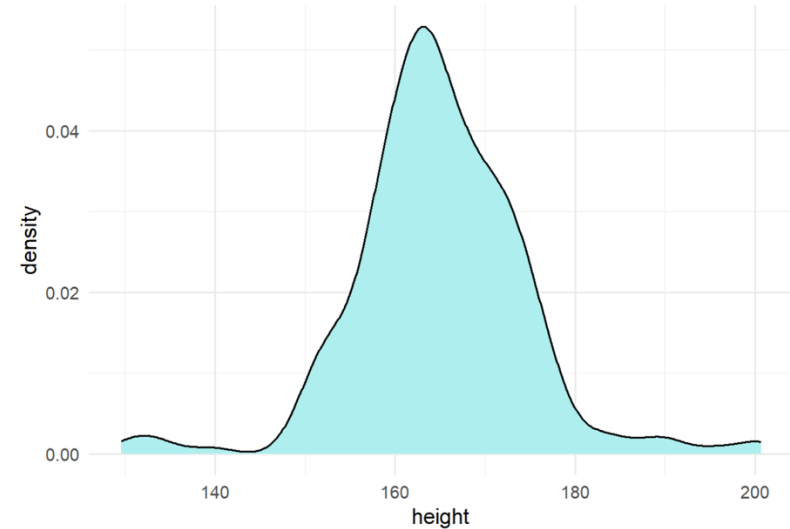
- The range of the data is from 125 to 210 cm with the majority between 150 and 180 cm
- The heights are close to symmetric around 163 cm

What information do we lose?

- Note that all values in one interval are treated the same when computing bin heights. Therefore, the histogram does not distinguish between 163 and 164 cm.

Smoothed density

```
heights |>
  filter(sex == "Female") |>
  ggplot(aes(height)) +
  geom_density(fill='paleturquoise',
               adjust=1)
  theme_minimal()
```



Smooth density plots communicate similar information:

- The distribution looks more “curvy” now: no sharp edges at the interval boundaries, local peaks are smoothed out. Why?

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n K_h(x - x_i) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

- The scale of the y-axis changed from counts to density, which guaranteed that the area under the curve sum up to 1.
 - For any interval, the area under the curve for that interval (integral) gives an approximation of the proportion of data.

See also: https://en.wikipedia.org/wiki/Kernel_density_estimation

adjust for geom_density

- (from left to right)

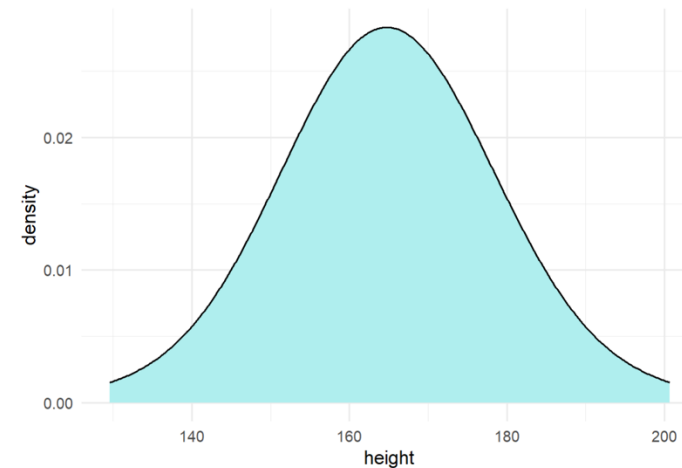
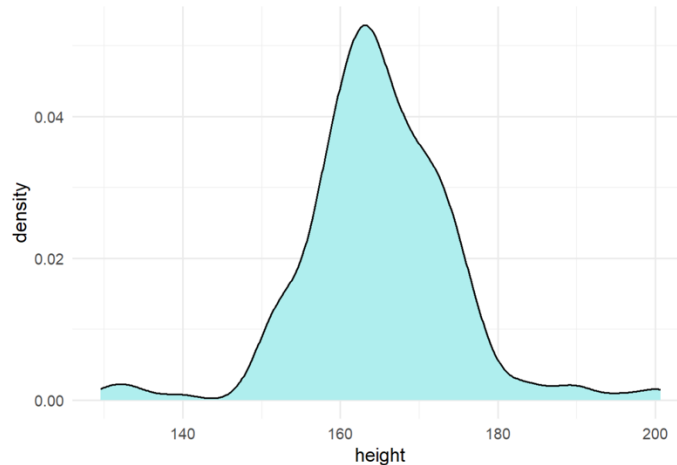
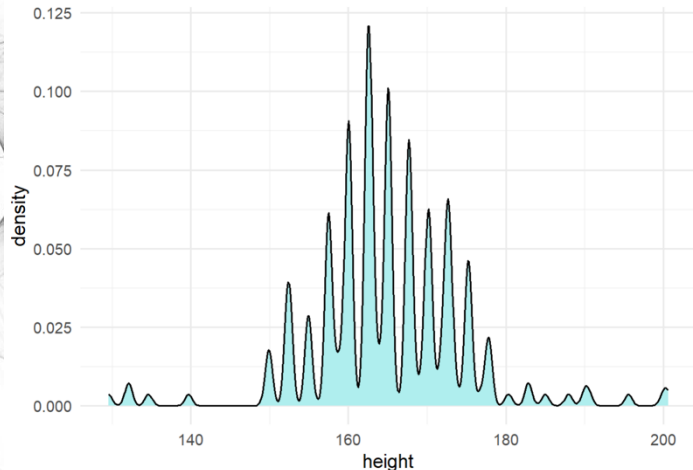
`adjust = 0.2`, `adjust = 1`,
`adjust = 5`

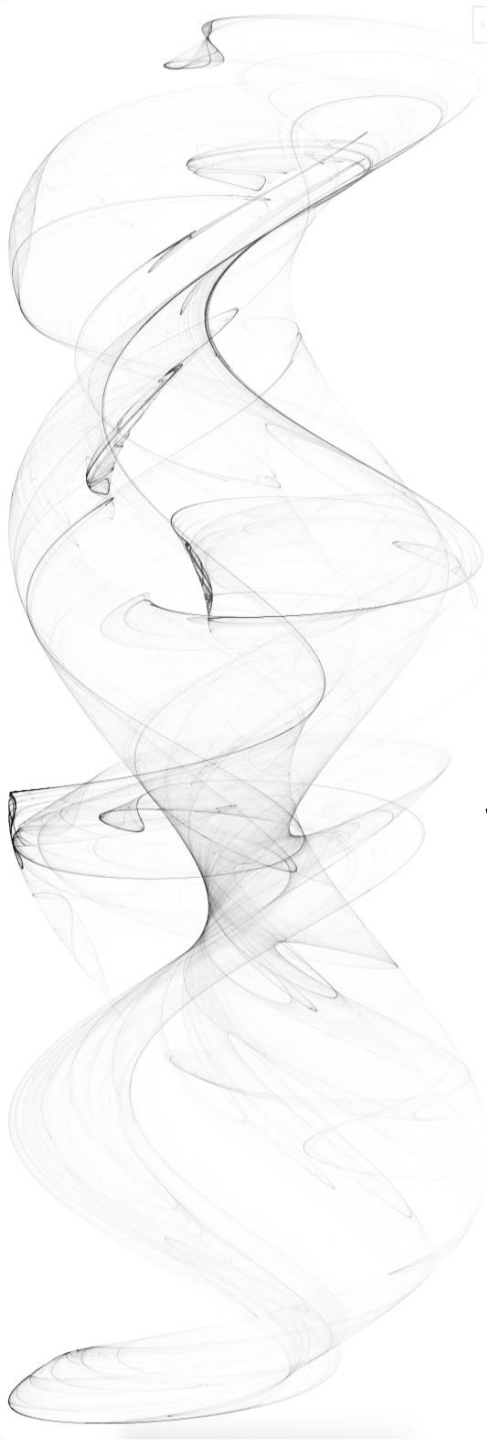
`bw`

The smoothing bandwidth to be used. If numeric, the standard deviation of the smoothing kernel. If character, a rule to choose the bandwidth, as listed in [stats::bw.nrd\(\)](#). Note that automatic calculation of the bandwidth does not take weights into account.

`adjust`

A multiplicative bandwidth adjustment. This makes it possible to adjust the bandwidth while still using a bandwidth estimator. For example, `adjust = 1/2` means use half of the default bandwidth.





What has been plotted?

In a histogram or a density plot

- One axis is numerical: different values of a variable (i.e., height)
- The other axis is also numerical: count/density of the variable taking different values.

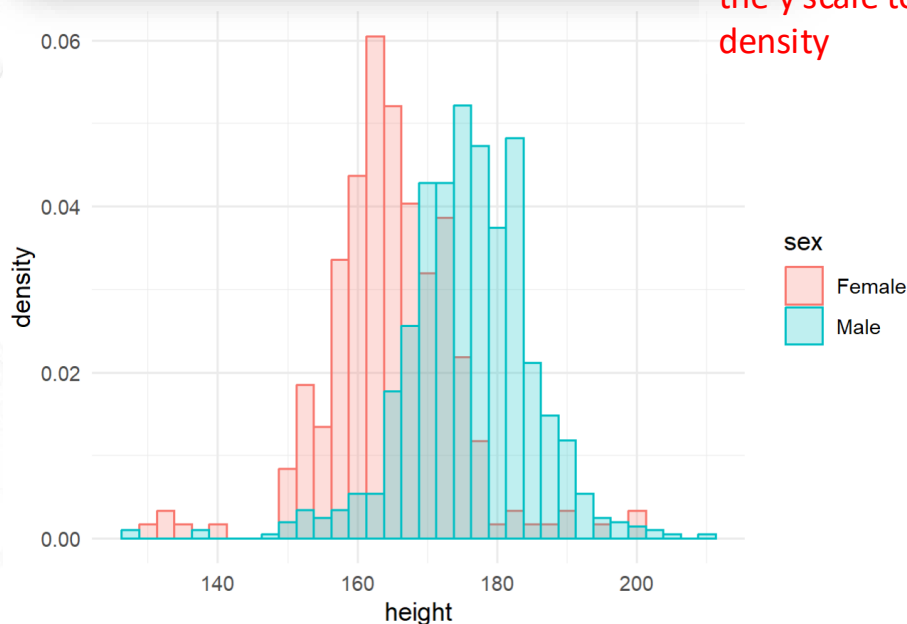
What if we want to compare two/multiple variables?

Compare distributions

Both histogram and density plots allow us to compare two distributions directly:

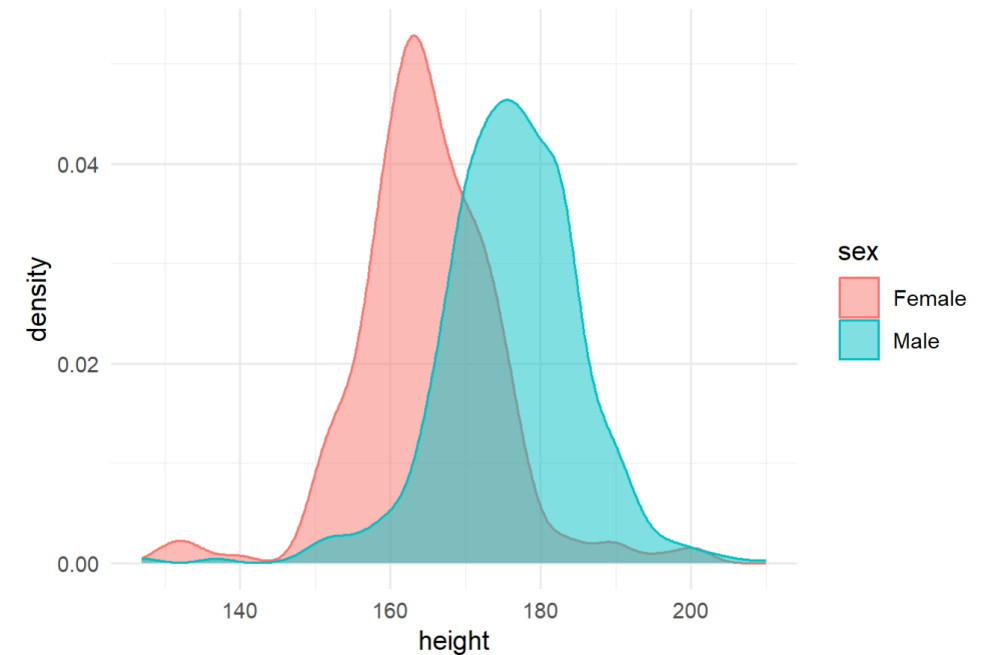
```
heights |>
  ggplot(aes(height, color = sex, fill = sex)) +
  geom_histogram(aes(y = after_stat(density)),
    binwidth = 2.5,
    alpha = 0.25,
    position = "identity") +
  theme_minimal()
```

Recall that
histogram use
counts by default,
this is changing
the y scale to
density



```
heights |>
  ggplot(aes(height, color = sex, fill = sex)) +
  geom_density(adjust = 1, alpha = 0.5) +
  theme_minimal()
```

Group observations
by "sex" and color
them differently



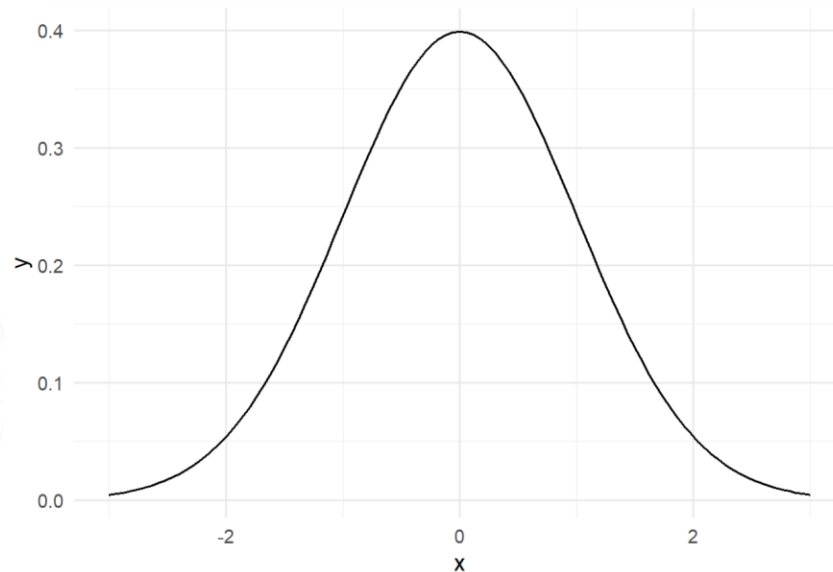
Density plot usually works better.

Normal/Gaussian distribution

We often see the usage of **average/mean** and **standard deviation** as summary statistics.

- Are these two numbers sufficient? Why?
- The **normal distribution**, also known as the bell curve or **Gaussian distribution**.

```
> ggplot(data.frame(x=c(-3, 3)), aes(x=x)) + stat_function(fun=dnorm) + theme_minimal()
```

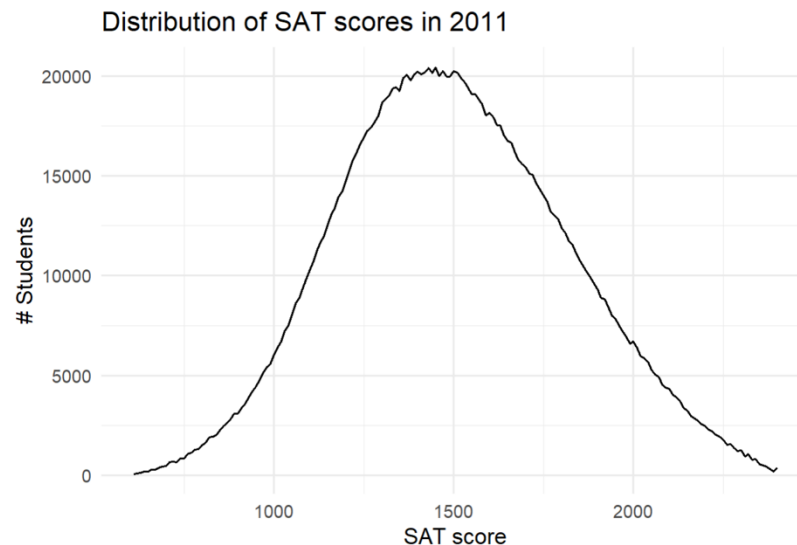


The plot shows a normal distribution with average 0 and SD 1

Normal/Gaussian distribution

The normal distribution is one of the most famous mathematical concepts in history. A reason for this is that **the distribution of many datasets can be approximated with normal distributions**. These include including gambling winnings, heights, weights, blood pressure, standardized test scores, and experimental measurement errors.

Normal distribution can be adapted to different datasets by just adjusting two numbers: average/mean, standard deviation (SD)



Data from: <https://secure-media.collegeboard.org/digitalServices/pdf/SAT-Percentile-Ranks-Composite-CR-M-W-2011.pdf>

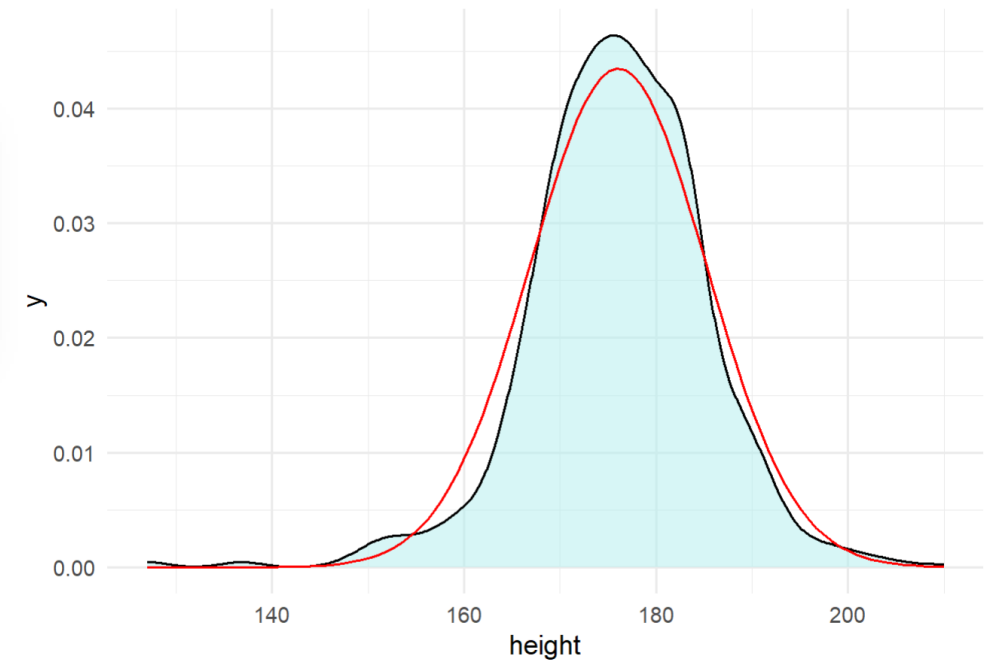


Normal/Gaussian distribution

To investigate whether the heights dataset complies normal distribution, let's overlay the heights distribution with a normal distribution

```
male_heights <- heights |> filter(sex == "Male")
m <- mean(male_heights$height)
sd <- sd(male_heights$height)

male_heights |>
  ggplot(aes(height)) +
  geom_density(adjust = 1, alpha = 0.5, fill="paleturquoise") +
  stat_function(fun = dnorm, args = list(mean = m, sd = sd), color="red") +
  theme_minimal()
```



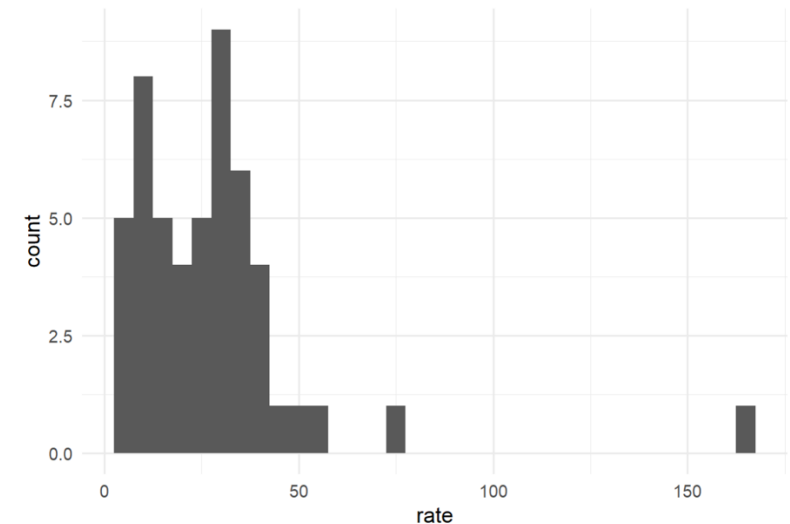
Boxplot

Percentiles are the values for which $p = 0.01, 0.02, \dots, 0.99$ of the data are less than or equal to that value, respectively.

We call, for example, the case of $p = 0.10$ the 10th percentile, which gives us a number for which 10% of the data is below. The most famous percentile is the 50th, also known as the **median**. Another special case that receives a name are the **quartiles**, which are obtained when setting $p = 0.25, 0.50$, and 0.75 .

To motivate boxplots, we will go back to the US murder data. Suppose we want to summarize the murder rate distribution. Using the data visualization skills we have learned, **we can quickly see that the normal approximation does not apply here**, more information is needed to describe the distribution of the data

```
murders <- murders |> mutate(rate = total / population * 1000000)
murders |>
  ggplot(aes(rate)) +
  geom_histogram(binwidth = 5) +
  theme_minimal()
```



Boxplot

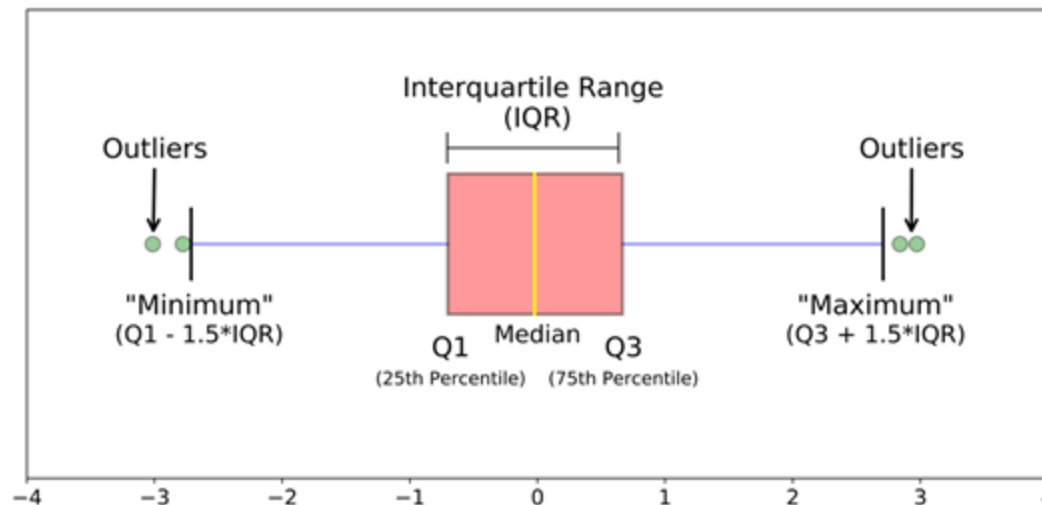
The boxplot provides a **five-number summary**:

- Minimum, Q1, median, Q3, maximum
- Components: the box (between Q1 & Q3), whiskers (minimum & maximum), outliers

How to define outliers?

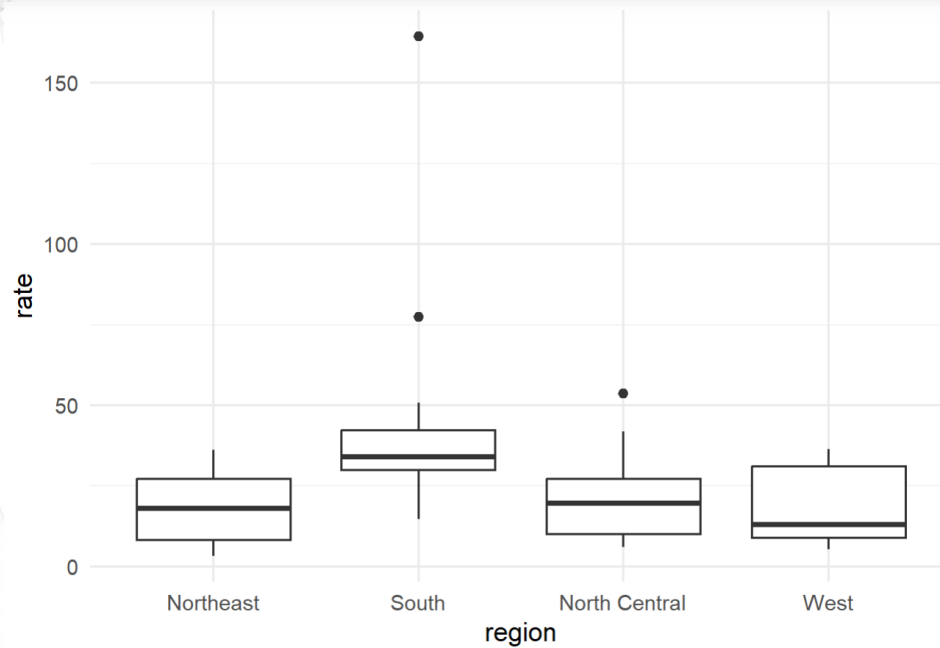
Interquartile range (IQR), also called the middle 50%, is a measure of statistical dispersion, being equal to the difference between 75th and 25th percentiles.

Outliers here are defined as observations that fall below $Q1 - 1.5 \text{ IQR}$ or above $Q3 + 1.5 \text{ IQR}$.

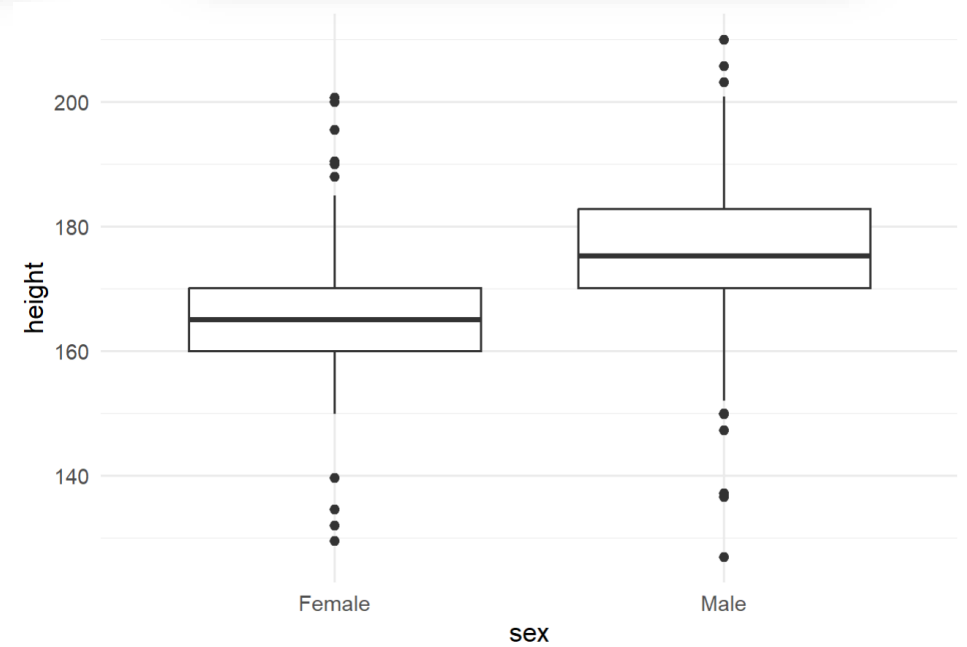


Boxplot

```
murders <- murders |> mutate(rate = total / population * 1000000)
murders |>
  ggplot(aes(x=region, y=rate)) +
  geom_boxplot() +
  theme_minimal()
```



```
heights |>
  ggplot(aes(x=sex, y=height)) +
  geom_boxplot() +
  theme_minimal()
```



Violin plot

```
heights |>  
  ggplot(aes(x=sex, y=height, color=sex, fill=sex)) +  
  geom_violin(alpha=0.5) +  
  theme_minimal()
```





What has been plotted?

In the boxplots and violin plot:

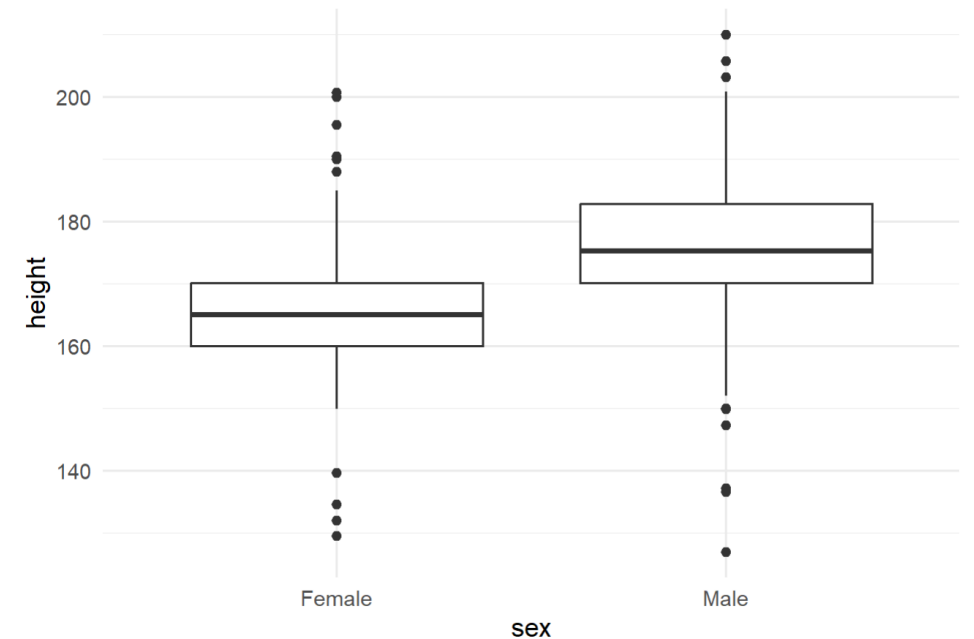
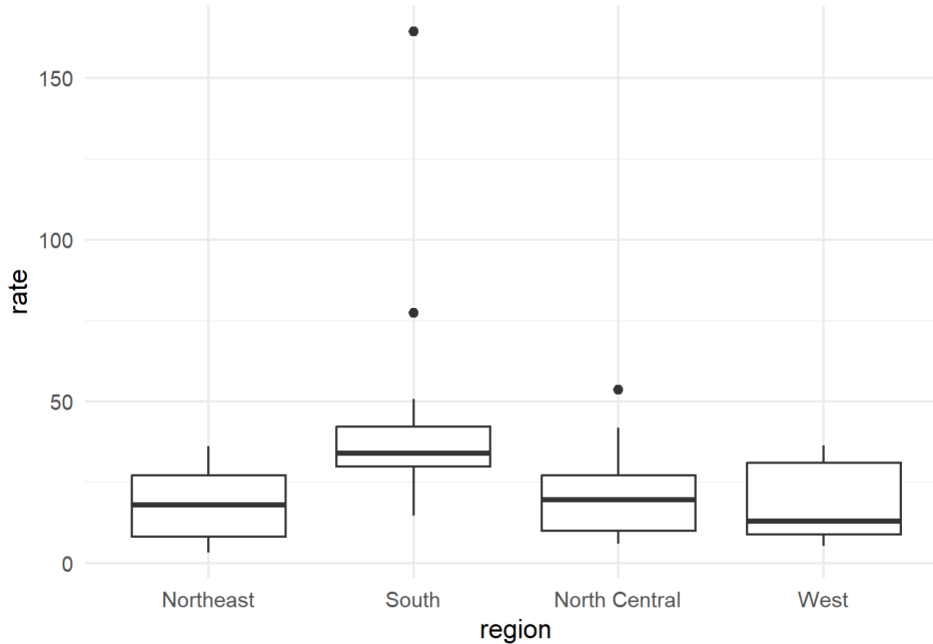
- One axis is categorical: different groups
- The other axis is numerical: a key property of the groups

Distributions are represented by the graphical elements (boxes, densities, etc.).

Stratification

In the previous examples, the murders dataset was grouped by regions, the heights dataset was grouped by gender

In data analysis we often divide observations into groups based on the values of one or more variables associated with those observations. We call this procedure stratification and refer to the resulting groups as strata.

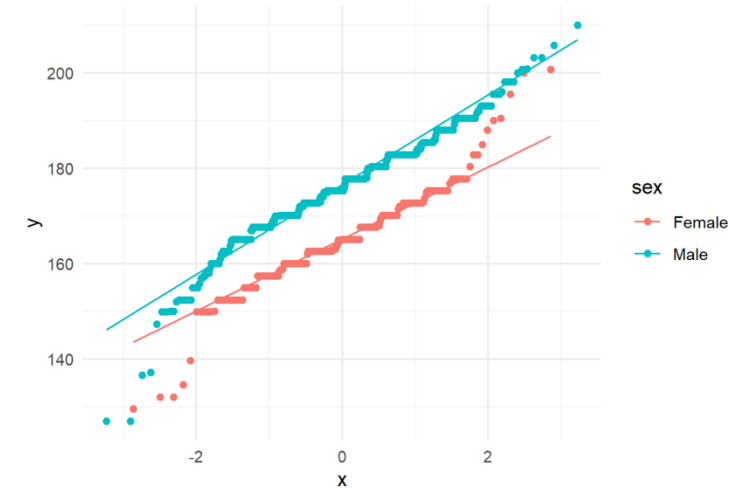


Q-Q plot

A Q-Q plot, short for “quantile-quantile” plot, is used to assess whether or not a set of data points potentially follow some theoretical distributions (e.g., normal distributions)

If the data is normally distributed, the points in a Q-Q plot will lie on a straight diagonal line

```
heights |>
  ggplot(aes(sample=height, color=sex)) +
  geom_qq() +
  geom_qq_line() +
  theme_minimal()
```



See also: <https://towardsdatascience.com/q-q-plots-explained-5aa8495426c0>



In this lecture

- Data visualization and its importance
- `ggplot2` basics
 - Layered construction of a graph
- Visualize data distributions
 - Bar plot
 - Histogram, density plot
 - Box plot